

Arhitectura calculatoarelor

Curs 8

8. Procesorul pipeline. Hazarduri

Cristian Vancea

<https://users.utcluj.ro/~vcristian/AC.html>

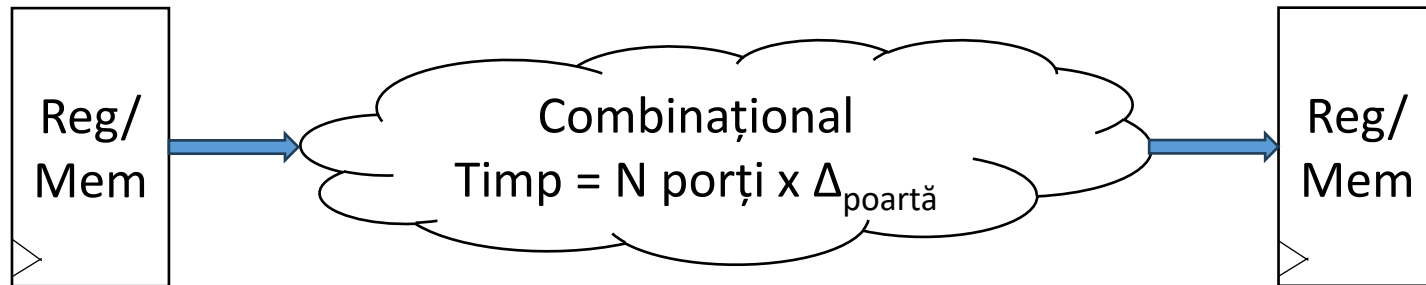
Objective

1. Proiectarea pipeline
2. Proiectarea pipeline pentru MIPS
 - 2.1 Operații în etajele MIPS pipeline
 - 2.2 Unitatea și semnalele de control
 - 2.3 Arhitectura completă
3. Hazarduri
4. Hazarduri – Sumar pentru laborator

8.1 Proiectarea pipeline

8.1.1 Motivație

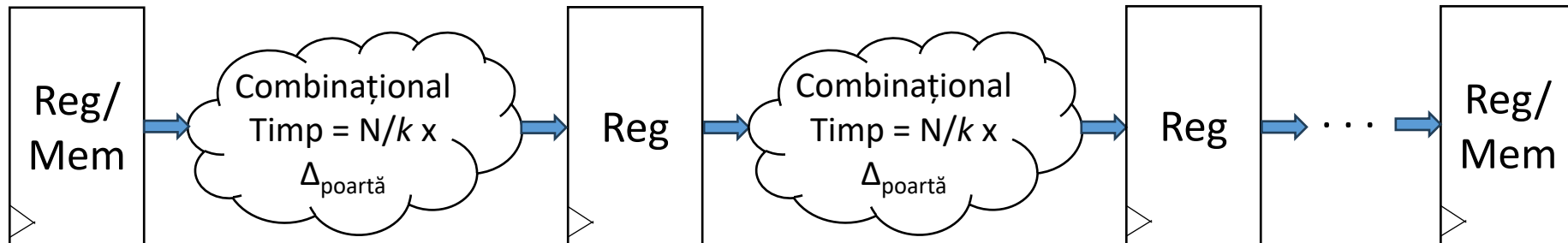
Dezavantaj la varianta cu ciclu unic: **perioadă de ceas mare datorată căii critice.**



Timp rezultat: $T_{rez} = T_{1ciclu} = N \times \Delta_p$, **Productivitate** (rezultate/timp): $Prod_{1ciclu} = 1/T_{rez} = 1/T_{1ciclu}$

Soluție

Secționarea căii critice cu registre => pipeline (linie de asamblare) = etaje de prelucrare separate prin registre conectate în serie; ieșirea unui etaj = intrare pentru următorul.



Pipeline cu k etaje

$$T_{rez(k)} = N/k \times \Delta_p = T_{1ciclu} / k, \quad Prod_k = 1 / T_{rez} = k / T_{1ciclu} = k \times Prod_{1ciclu}$$

8.1 Proiectarea pipeline

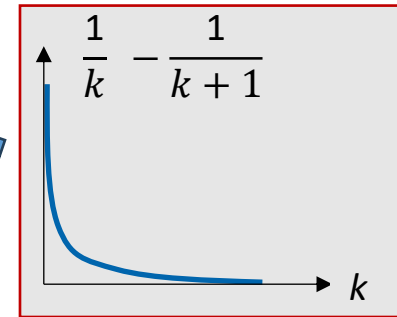
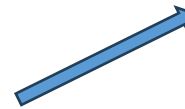
8.1.2 Determinarea unui k optim, considerând întârzierea pe registru = Δ_{REG} .

$$\text{Prod}_k = \frac{1}{\frac{T_{1\text{ciclu}}}{k} + (k-1) \times \Delta_{\text{REG}}} \quad \text{Prod}_{k+1} = \frac{1}{\frac{T_{1\text{ciclu}}}{k+1} + k \times \Delta_{\text{REG}}} \Rightarrow$$

Când începe să scadă productivitatea crescând factorul k ?

Răspuns: $\text{Prod}_{k+1} < \text{Prod}_k$, dacă $\Delta_{\text{REG}} > T_{1\text{ciclu}} \times \left(\frac{1}{k} - \frac{1}{k+1} \right)$

$\Rightarrow k_{\text{optim}}$ depinde de Δ_{REG} și $T_{1\text{ciclu}}$



Apar 2 cazuri:

1. $k < k_{\text{optim}} \Rightarrow$ arhitectura *underpipelined*
2. $k > k_{\text{optim}} \Rightarrow$ arhitectura *overpipelined*

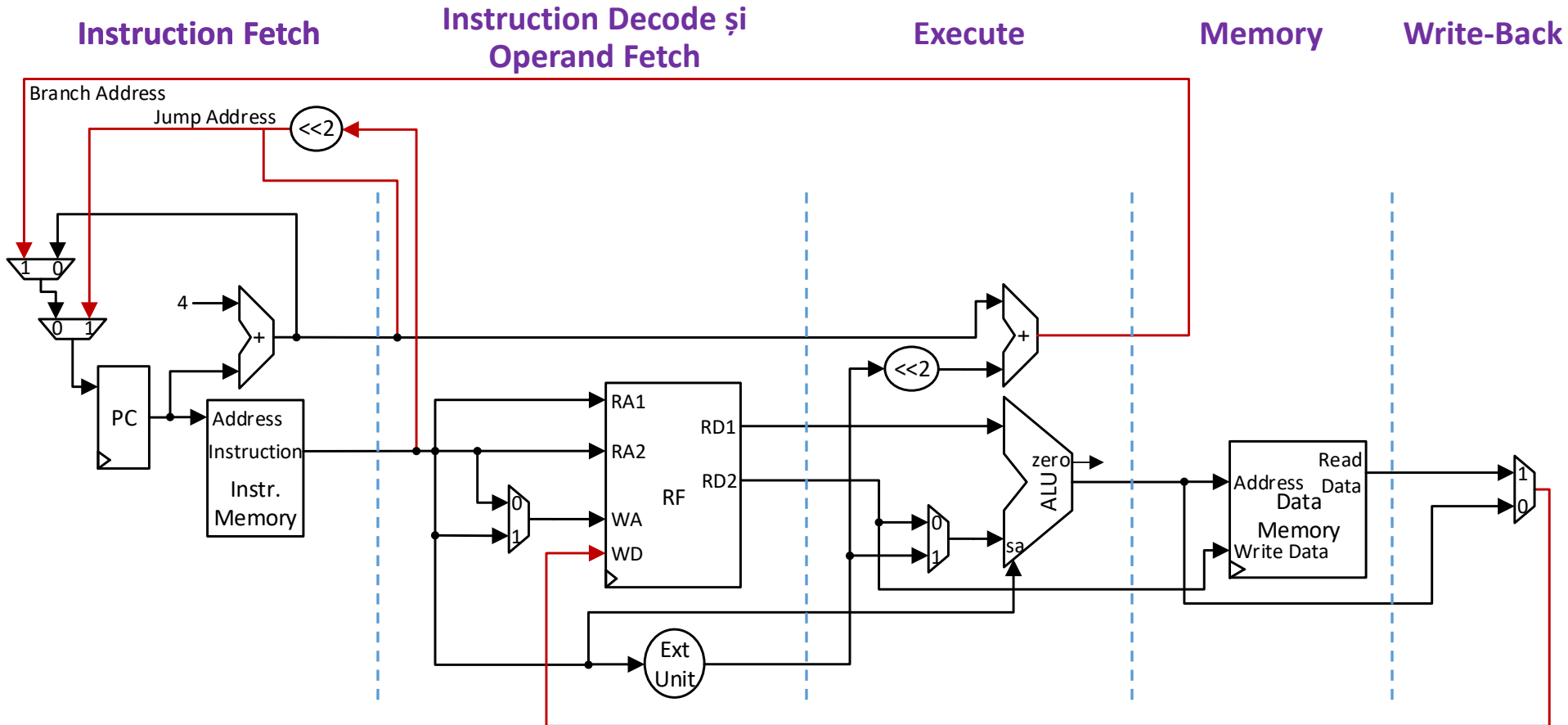
8.1 Proiectarea pipeline

8.1.3 Caracteristici

- Se **exploatează paralelismul la nivel de instrucțiuni** (Instruction Level Parallelism - ILP) => faze diferite ale instrucțiunilor sunt executate în paralel, în etaje diferite.
- Nu scade timpul de execuție al unei instrucțiuni, dar crește productivitatea => scade timpul de execuție al programului, cu factorul k = nr. de etaje, în cazul ideal. **Observații:**
 - Etajele nu sunt perfect balansate => viteza este **limitată de etajul cel mai lent**;
 - Pot apărea întârzieri suplimentare necesare pentru a evita hazardurile;
 - Apar și timpii de setup T_{setup} și hold T_{hold} la scrierea în registrele pipeline.
- Execuția în pipeline este **invizibilă pentru programator**.
- La proiectarea pipeline **se urmărește**:
 - Balansarea etajelor ca timp de întârziere;
 - Eliminarea hazardurilor.
- **Avantajele** ISA MIPS la proiectarea pipeline:
 - Instrucțiunile au lungime fixă → facilitează implementarea pipeline;
 - Număr redus de formate de instrucțiune cu locații fixe pentru registre în instrucțiuni;
 - Operații numai pe registre, fără operanzi din memorie; accesul la memorie este limitat la instrucțiuni de load și store.
- **Dificultățile** în execuția pipeline sunt date de hazarduri: structurale, de date și de control.

8.2 Proiectarea pipeline pentru MIPS

Se pornește de la arhitectura MIPS cu ciclu unic (calea de date)

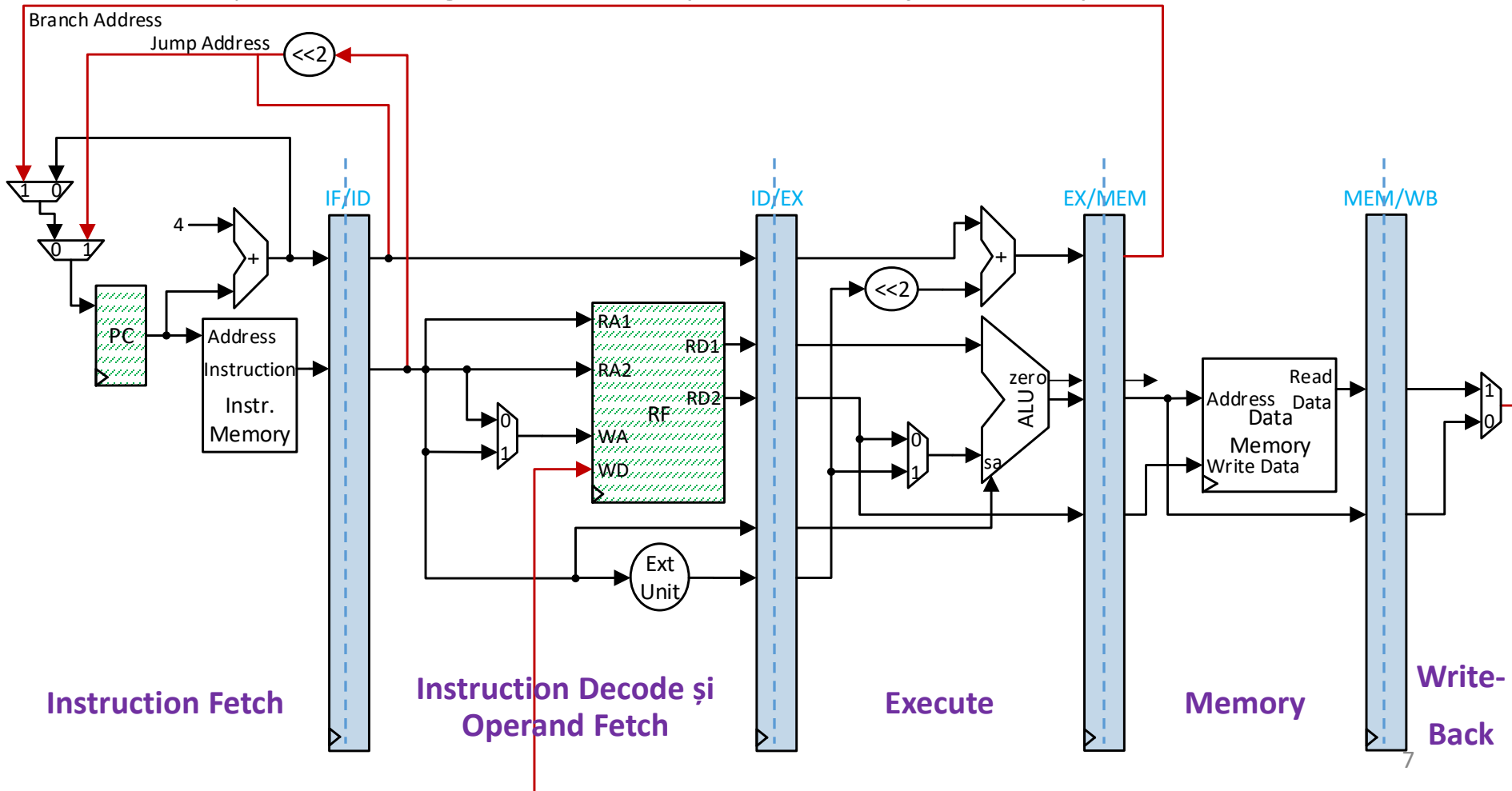


- Se secționează în cele 5 etaje de execuție: IF, ID&OF, EX, MEM, WB => se vor executa până la 5 instrucțiuni în pipeline.
- Fluxul de date este de la stânga la dreapta. **Obs:** Avem Write-Back în RF și revenire a adreselor de salt către multiplexoarele de la PC => **aceste linii vor ocoli registrele.**

8.2 Proiectarea pipeline pentru MIPS

Se pornește de la arhitectura MIPS cu ciclu unic (calea de date) (cont)

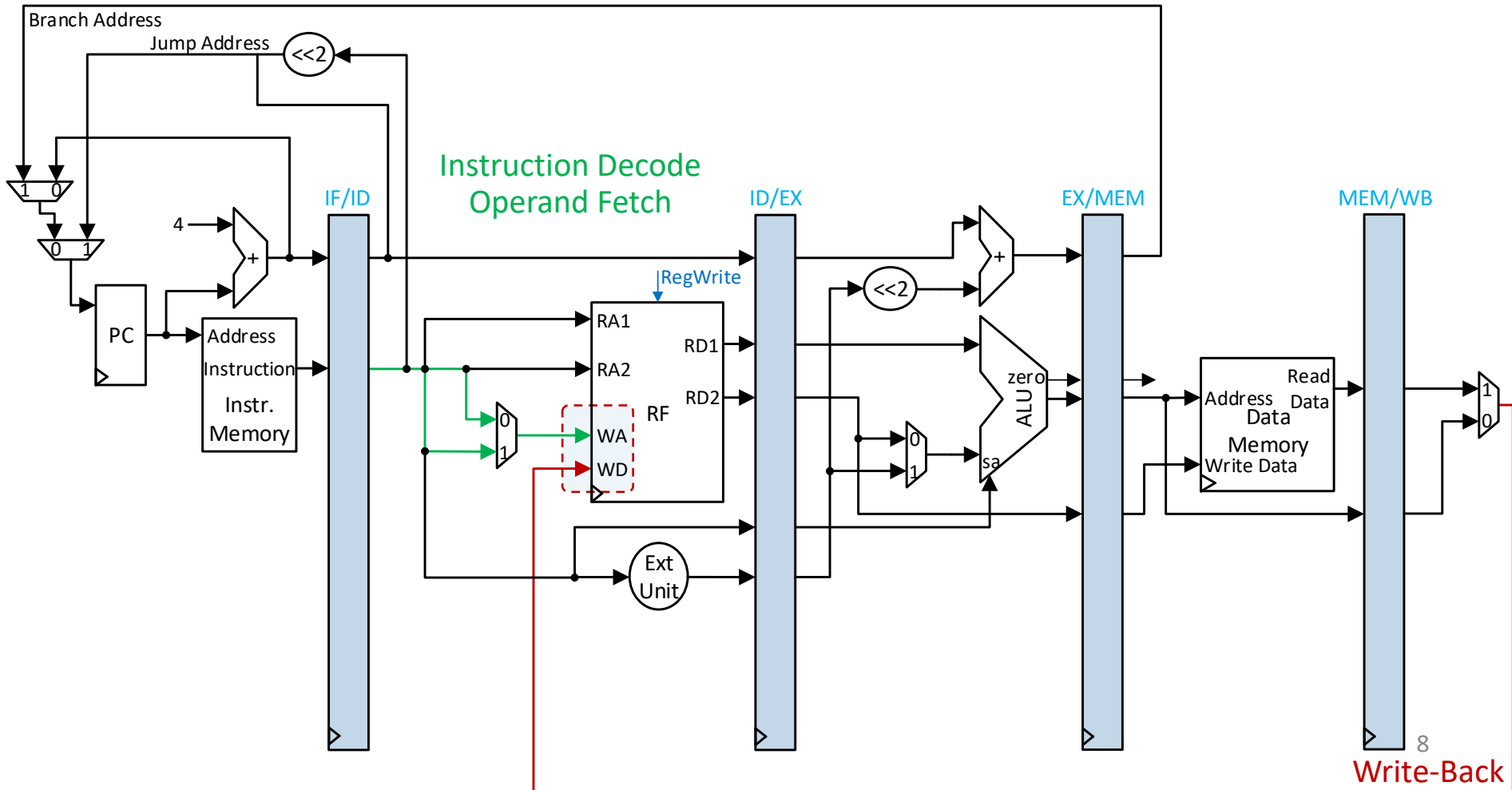
- Se introduc registrele IF/ID, ID/EX, EX/MEM, MEM/WB între etaje, cu scriere pe frontul ascendent → vor transfera datele și semnalele de control între etaje.
- Notă:** PC și RF sunt registre, dar fac parte din etajele IF, respectiv ID&OF.



8.2 Proiectarea pipeline pentru MIPS

Prolemă

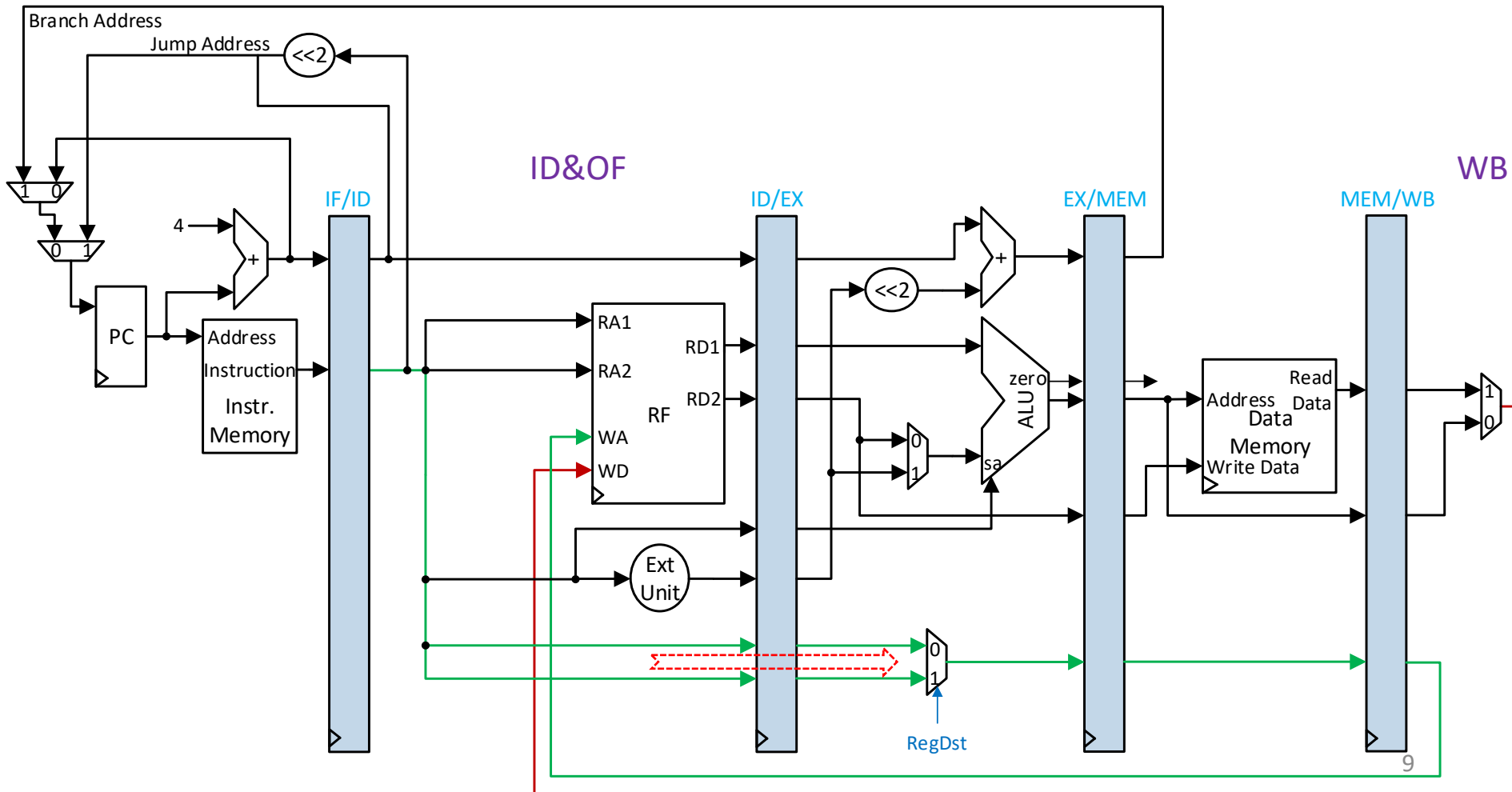
- WA și WD provin de la instrucțiuni separate, aflate în faze diferite de execuție, dar au efect simultan asupra RF: scrierea folosește date din WB și adresa din ID&OF.
- **Notă:** WA nu este necesar în etajul IF&OF, deci poate fi amânat până în etajul WB.



8.2 Proiectarea pipeline pentru MIPS

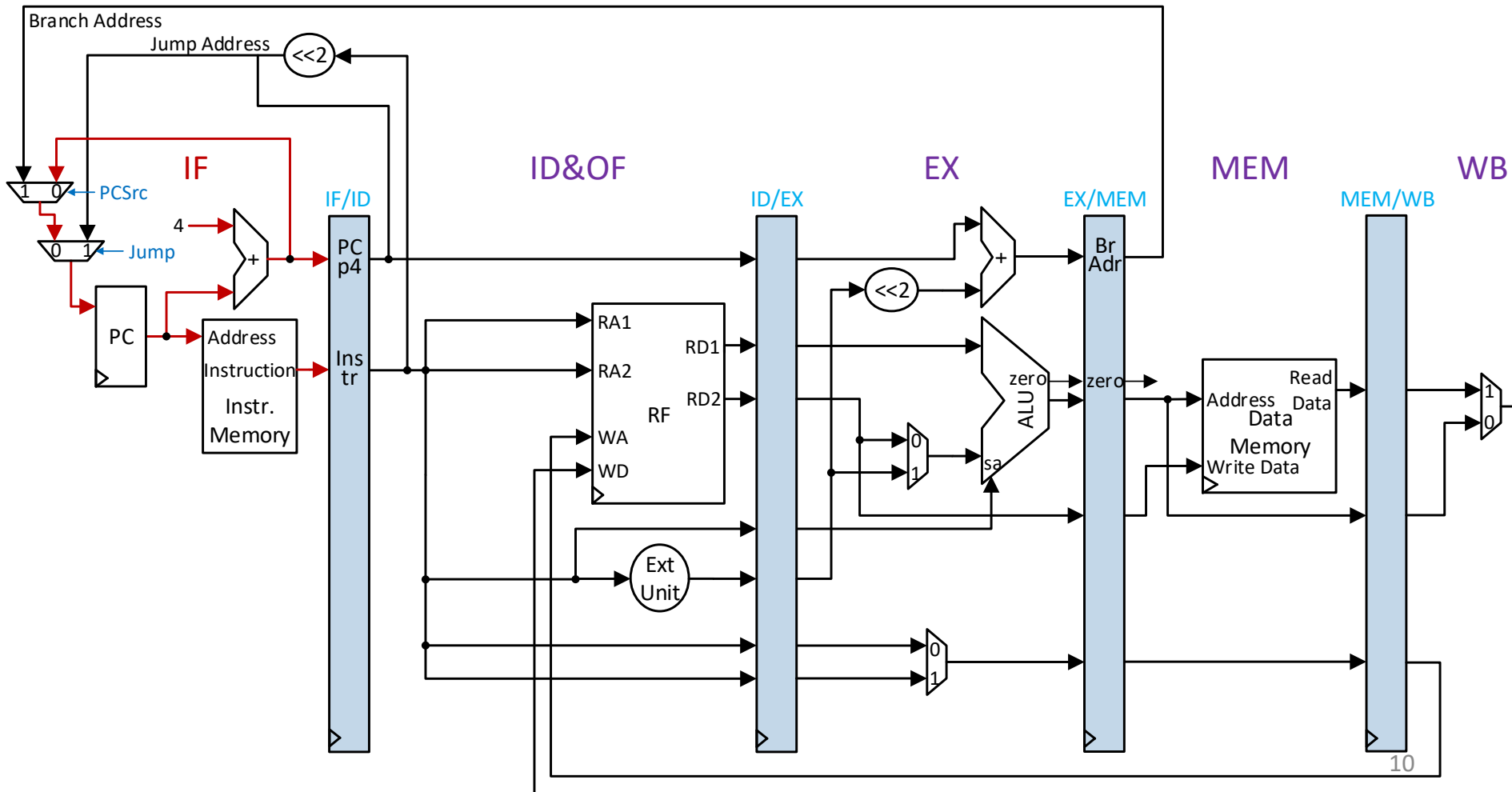
Soluție

- Adresa și datele traversează împreună arhitectura pipeline până în etajul WB.
- Multiplexorul cu selecția RegDst se mută în etajul EX (din motive ce țin de hazard).



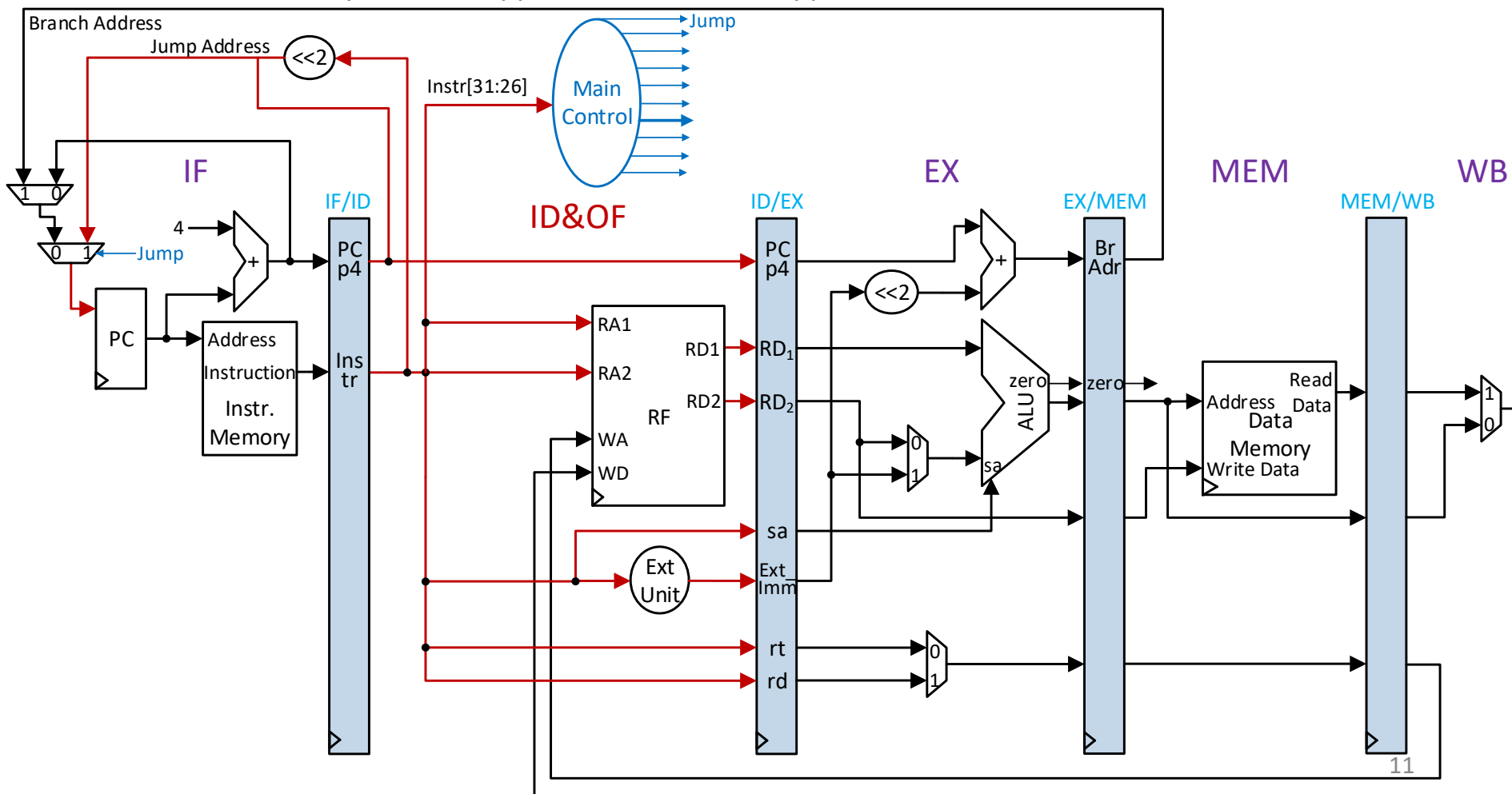
8.2.1 Operații în etajele MIPS pipeline – Instruction Fetch

- **IF/ID**.Instr $\leftarrow$ IM[PC] (se încarcă instrucțiunea din memoria de instrucțiuni)
- **IF/ID**.PCp4 $\leftarrow$ PC+4 (se transmite PC+4 spre etajul următor)
- PC $\leftarrow$ PC+4



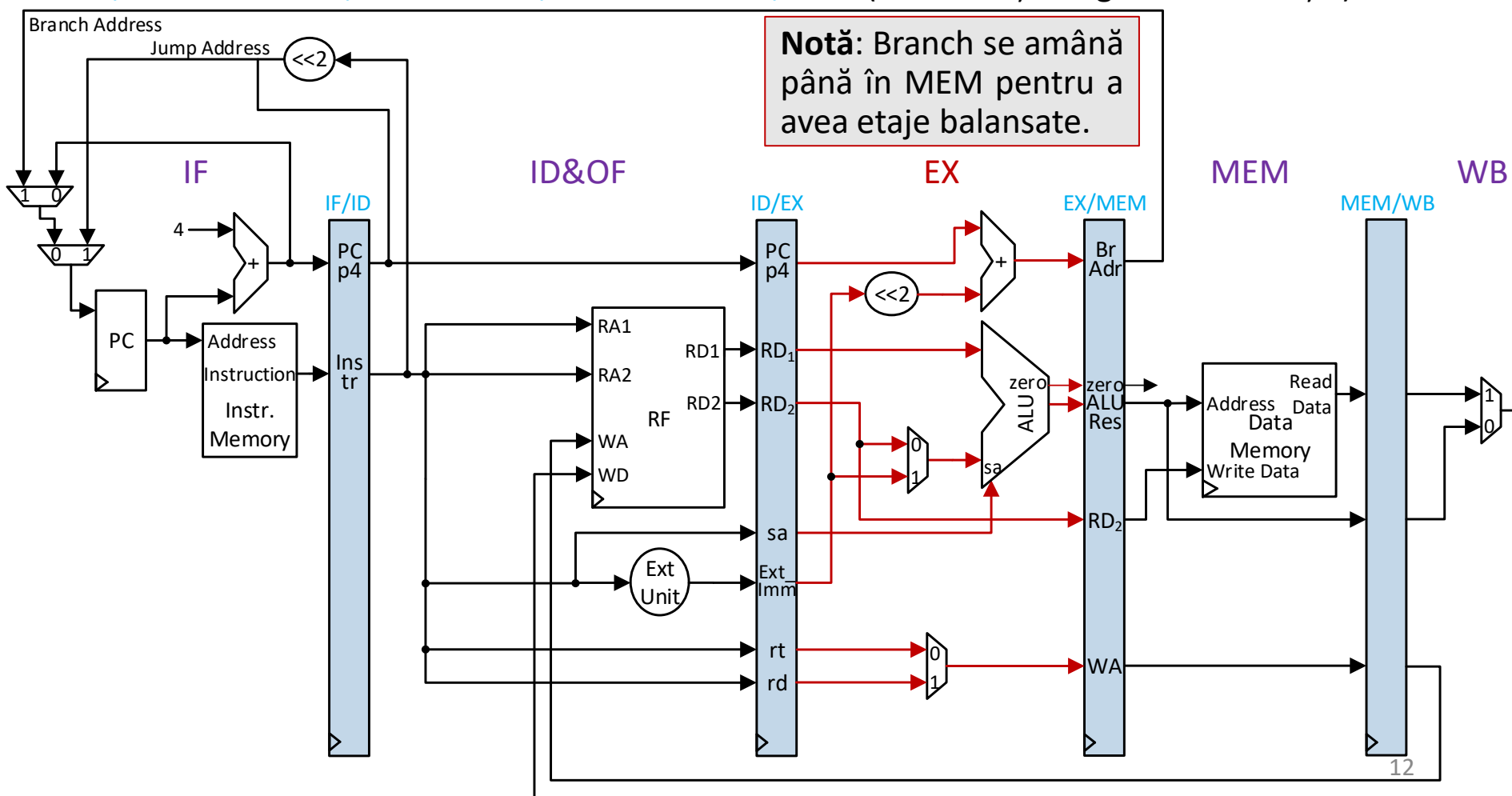
8.2.1 Operații în etajele MIPS pipeline – Instruction Decode și Operand Fetch

- $ID/EX.RD_1 \leftarrow RF[rs]$, $ID/EX.RD_2 \leftarrow RF[rt]$ (se încarcă operanzii; citirea din RF este asincronă)
- $ID/EX.Ext_Imm \leftarrow S/Z_Ext(imm)$ (se extinde imediatul)
- $ID/EX.rt \leftarrow rt$, $ID/EX.rd \leftarrow rd$, $ID/EX.sa \leftarrow sa$ (se transmit câmpurile rt , rd și sa la etajul următor)
- $ID/EX.PCp4 \leftarrow IF/ID.PCp4$ (se transmite PC+4 la etajul următor)
- la J: $PC \leftarrow IF/ID.PCp4[31:28] || (IF/ID.Instr[26:0] || 00)$



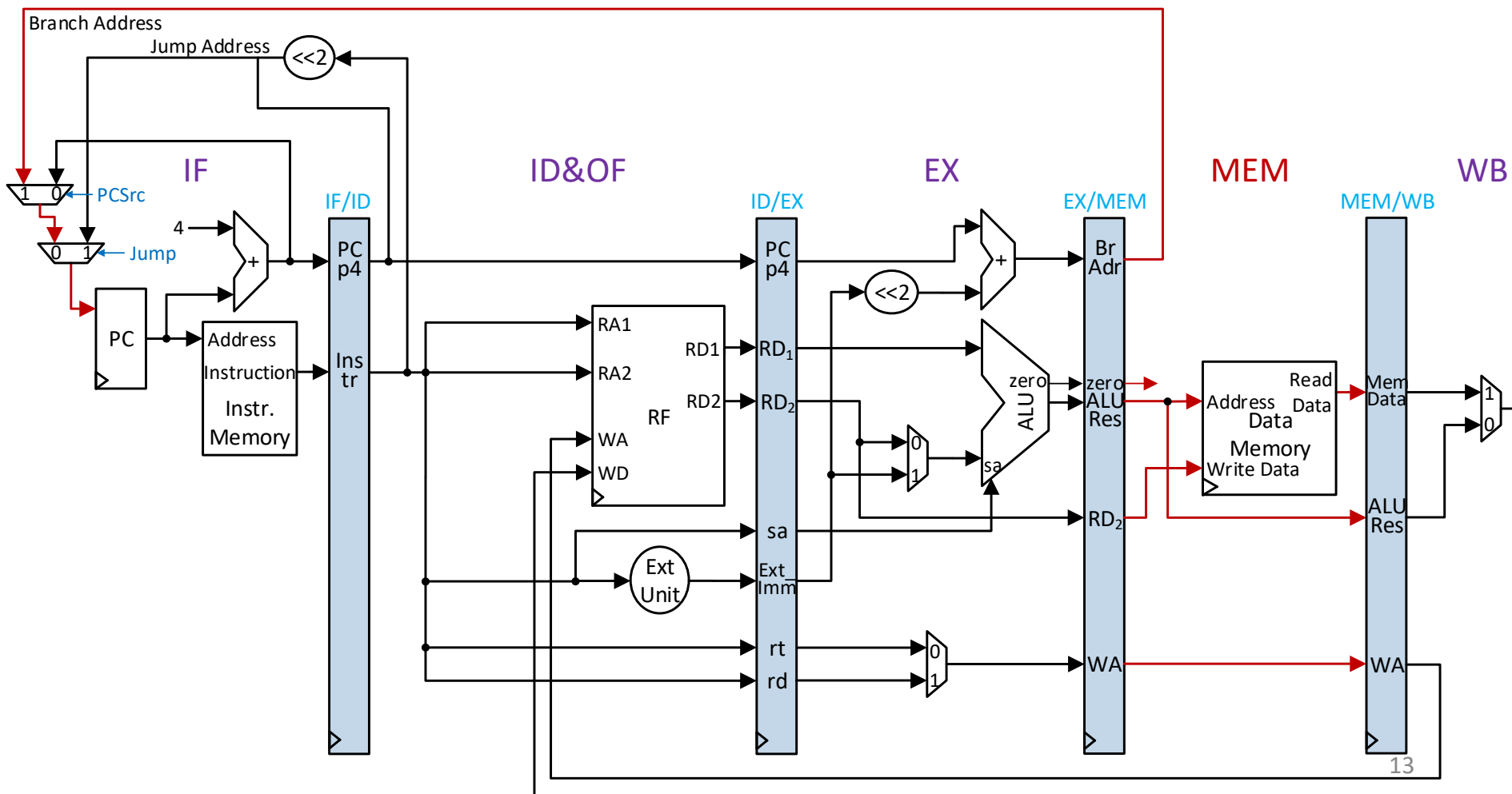
8.2.1 Operații în etajele MIPS pipeline – **Execute**

- $EX/MEM.BrAdr \leftarrow ID/EX.PCp4 + (ID/EX.Ext_Imm \ll 2)$ (se calculează adresa de branch)
- $EX/MEM.zero \leftarrow zero$ (se transmite zero la etajul următor pentru branch)
- $EX/MEM.ALURes \leftarrow ID/EX.RD_1 \text{ op } ID/EX.RD_2$ sau $EX/MEM.ALURes \leftarrow ID/EX.RD_1 \text{ op } ID/EX.Ext_Imm$
- $EX/MEM.RD_2 \leftarrow ID/EX.RD_2$ (se transmite $RF[rt]$ la etajul următor pentru scriere în memorie)
- $EX/MEM.WA \leftarrow ID/EX.rd$ sau $EX/MEM.WA \leftarrow ID/EX.rt$ (se stabilește registrul destinație)



8.2.1 Operații în etajele MIPS pipeline – Memory

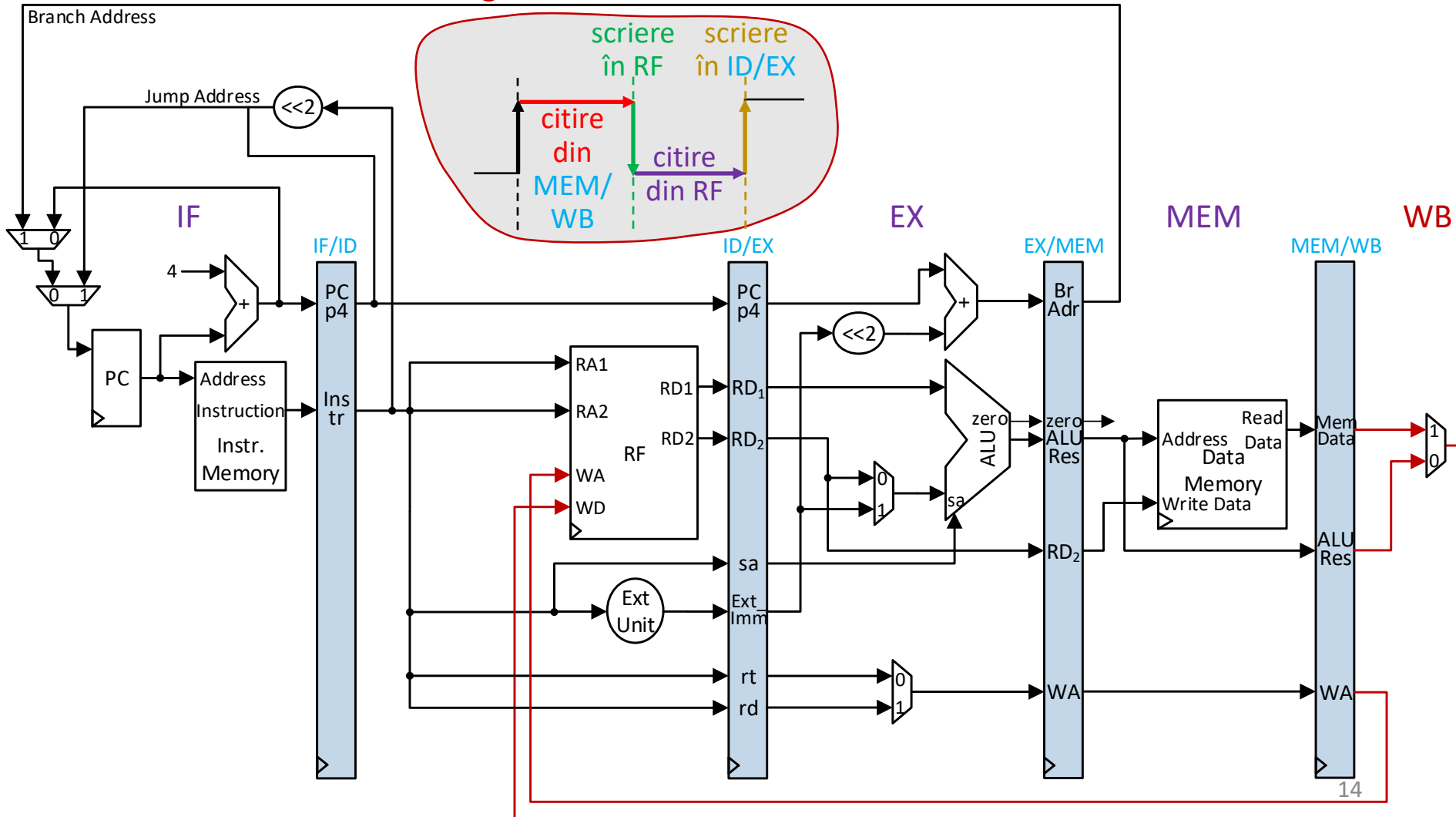
- la LD: $\text{MEM/WB.MemData} \leftarrow \text{DM}[\text{EX/MEM.ALURes}]$ (se citește din memoria de date)
- la SW: $\text{DM}[\text{EX/MEM.ALURes}] \leftarrow \text{EX/MEM.RD}_2$ (se scrie RF[rt] în memoria de date)
- $\text{MEM/WB.ALURes} \leftarrow \text{EX/MEM.ALURes}$ (se transmite rezultatul ALU la etajul următor)
- $\text{MEM/WB.WA} \leftarrow \text{EX/MEM.WA}$ (se transmite registrul destinație la etajul următor)
- la BEQ: $\text{EX/MEM.zero} \rightarrow \text{PC} \leftarrow \text{EX/MEM.BrAdr}$



8.2.1 Operații în etajele MIPS pipeline – Write-Back

- la LW: $RF[MEM/WB.WA] \leftarrow MEM/WB.MemData$ (se scrie în RF data citită din memorie)
- la operații aritmetice-logice: $RF[MEM/WB.WA] \leftarrow MEM/WB.ALURes$ (se scrie în RF rezultatul ALU)

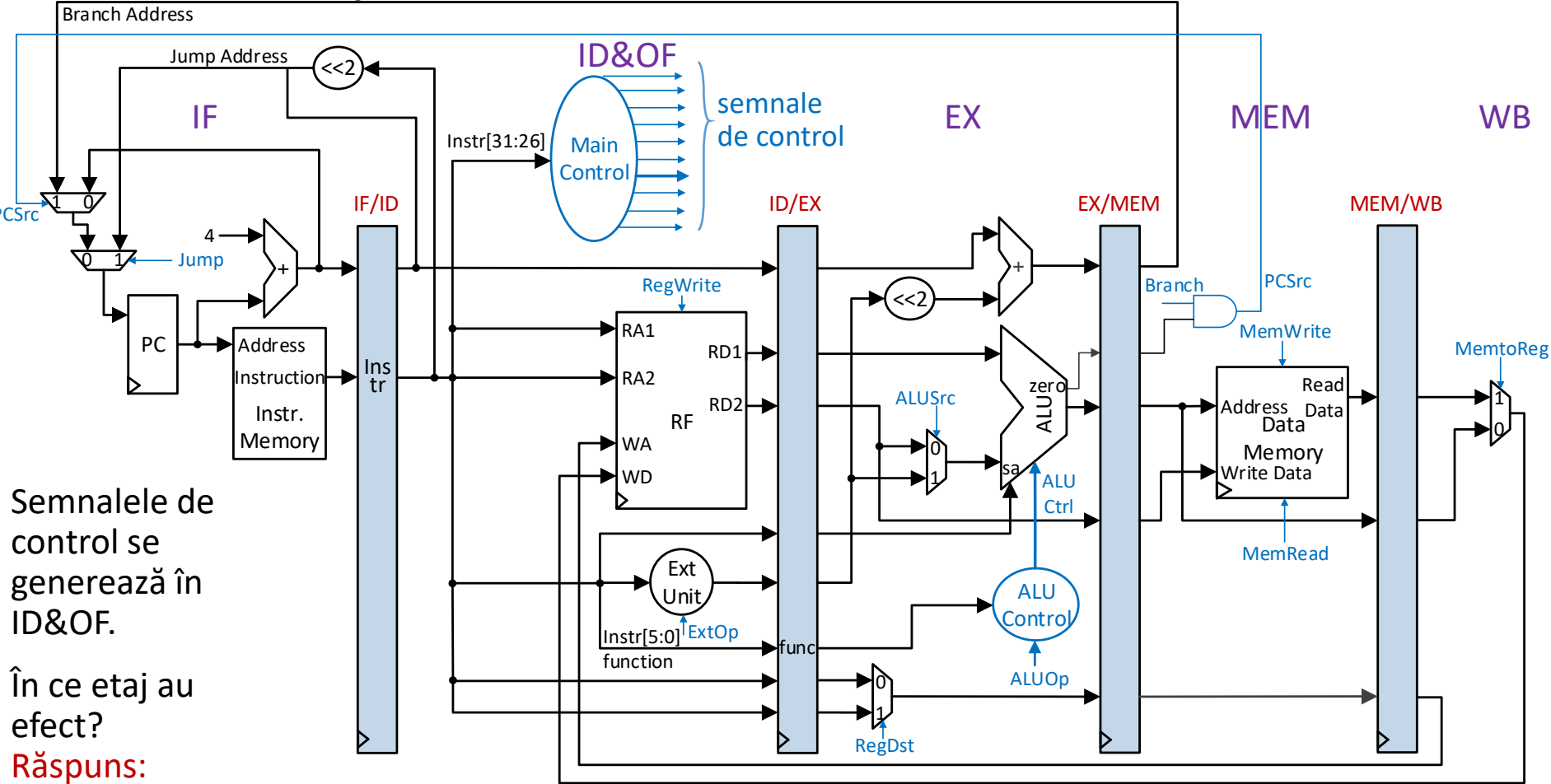
Notă: Scrierea în RF se face pe frontul descendent pentru a permite citirea valorii actualizate, în același ciclu.



8.2.1 Operații în etajele MIPS pipeline – Rezumat

	Tip R	Tip I Aritm./Logic	LW	SW	BEQ	J
IF	$IF/ID.Instr \leftarrow IM[PC], IF/ID.PCp4 \leftarrow PC+4$ Jump $\rightarrow PC \leftarrow IF/ID.PCp4[31:28] (IF/ID.Instr[26:0] 00)$ sau (Branch & $EX/MEM.zero$) $\rightarrow PC \leftarrow EX/MEM.BrAdr$ sau $PC \leftarrow PC+4$					
ID	$ID/EX.RD_2 \leftarrow RF[rt]$ $ID/EX.sa \leftarrow sa$	$ID/EX.Ext_Imm \leftarrow S/Z_Ext(imm)$	$ID/EX.RD_1 \leftarrow RF[rs]$ $ID/EX.Ext_Imm \leftarrow S_Ext(imm)$		$ID/EX.RD_2 \leftarrow RF[rt]$ $ID/EX.PCp4 \leftarrow IF/ID.PCp4$	$PC \leftarrow IF/ID.PCp4[31:28] (IF/ID.Instr[26:0] 00)$
EX	$EX/MEM.ALURes \leftarrow ID/EX.RD_1 \text{ op } ID/EX.RD_2$	$EX/MEM.ALURes \leftarrow ID/EX.RD_1 \text{ op } ID/EX.Ext_Imm$	$EX/MEM.ALURes \leftarrow ID/EX.RD_1 + ID/EX.Ext_Imm$		$EX/MEM.zero \leftarrow zero$ $EX/MEM.BrAdr \leftarrow ID/EX.PCp4 + (ID/EX.Ext_Imm << 2)$	
MEM	$MEM/WB.ALURes \leftarrow EX/MEM.ALURes$		$MEM/WB.MemData \leftarrow DM[EX/MEM.ALURes]$	$EX/MEM.WA \leftarrow ID/EX.rd \text{ sau } EX/MEM.WA \leftarrow ID/EX.rt$	$EX/MEM.RD_2 \leftarrow ID/EX.RD_2$	$EX/MEM.zero \rightarrow PC \leftarrow EX/MEM.PCp4$
WB	$MEM/WB.WA \leftarrow EX/MEM.WA$					
	$RF[MEM/WB.WA] \leftarrow MEM/WB.ALURes$		$RF[MEM/WB.WA] \leftarrow MEM/WB.MemData$			

8.2.2 Unitatea și semnalele de control



Semnalele de control se generează în ID&OF.

În ce etaj au efect?

Răspuns:

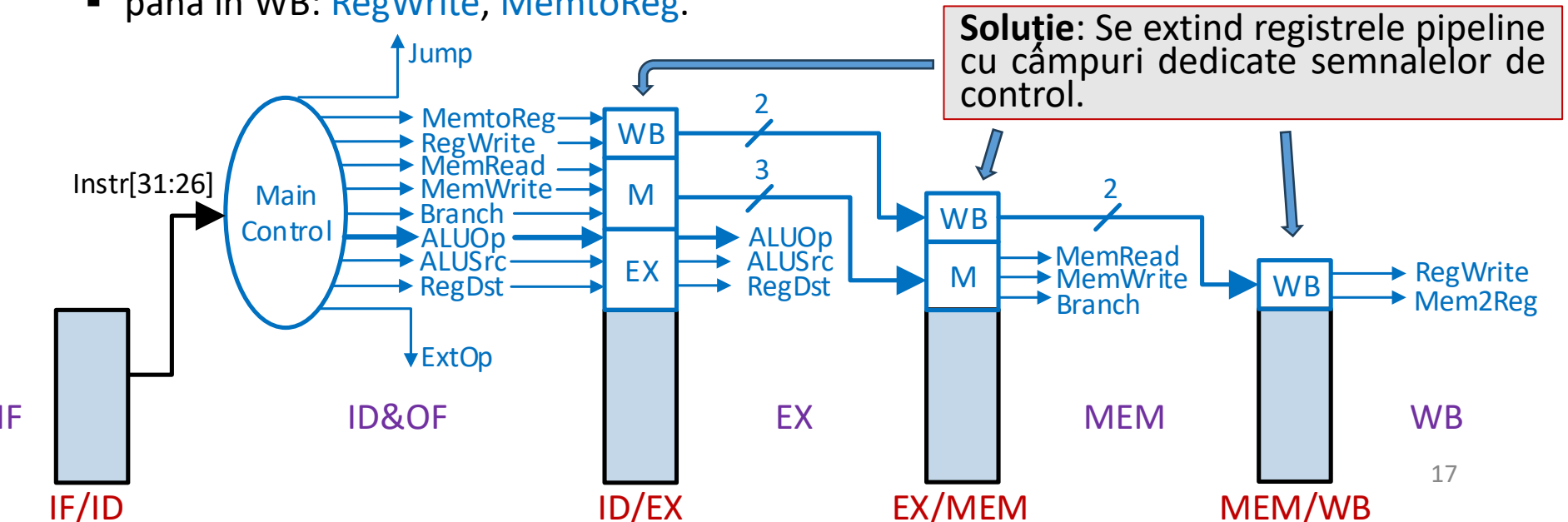
Etaj	ID&OF		EX			MEM			WB	
Instrucțiune	ExtOp	Jump	RegDst	ALUOp	ALUSrc	Branch	MemWrite	MemRead	RegWrite	MemtoReg
tip R	x	0	1	10 (R)	0	0	0	0	1	0
ORI	0	0	0	11 ()	1	0	0	0	1	0
ADDI	1	0	0	00 (+)	1	0	0	0	1	0
LW	1	0	0	00 (+)	1	0	0	1	1	1
SW	1	0	x	00 (+)	1	0	1	0	0	x
BEQ	1	0	x	01 (-)	0	1	0	0	0	16 x
J	x	1	x	xx	x	x	0	0	0	x

8.2.2 Unitatea și semnalele de control (cont)

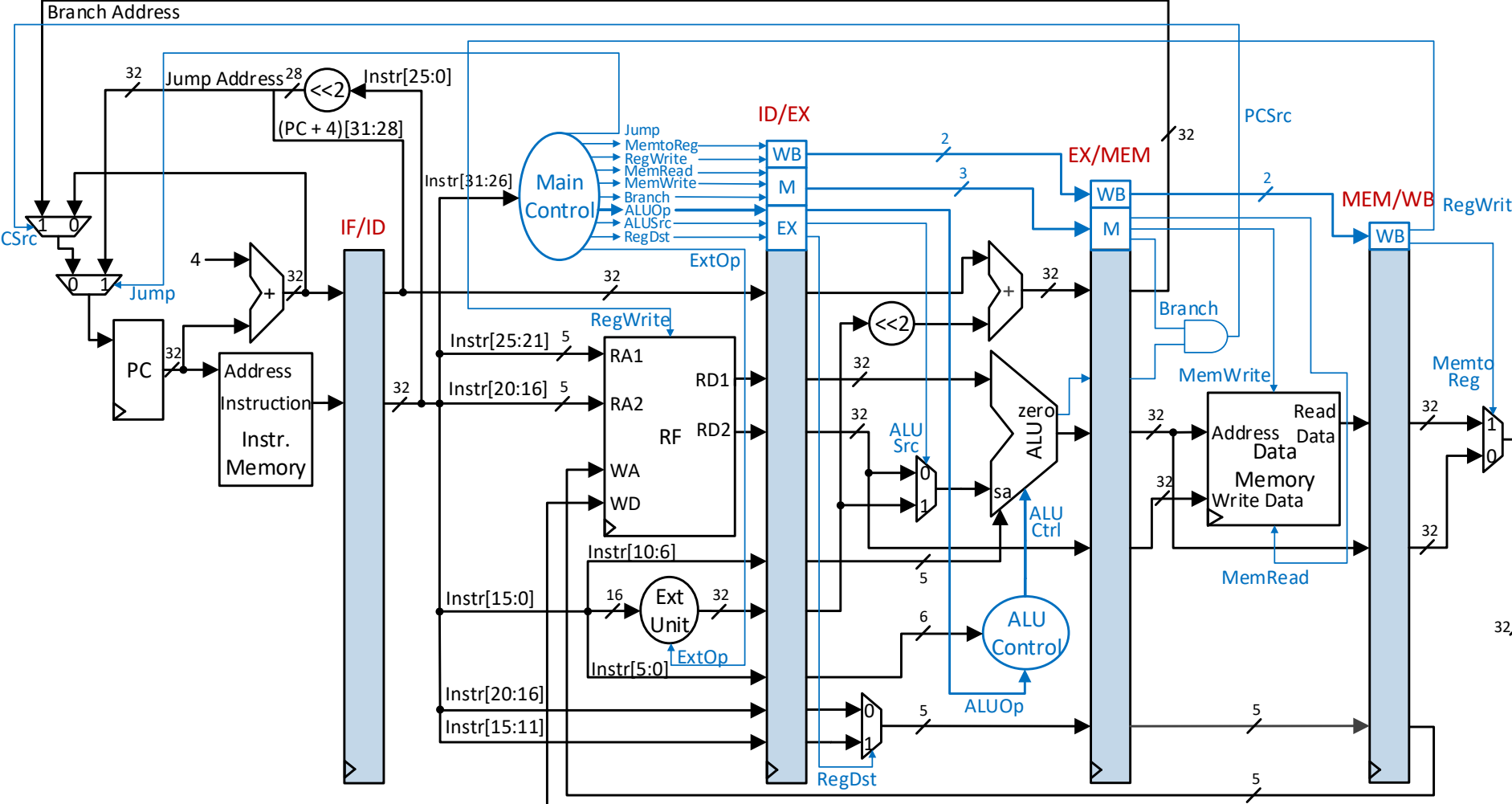
Etaj	ID&OF		EX			MEM			WB	
Instrucțiune	ExtOp	Jump	RegDst	ALUOp	ALUSrc	Branch	MemWrite	MemRead	RegWrite	MemtoReg
tip R	x	0	1	10 (R)	0	0	0	0	1	0
ORI	0	0	0	11 (I)	1	0	0	0	1	0
ADDI	1	0	0	00 (+)	1	0	0	0	1	0
LW	1	0	0	00 (+)	1	0	0	1	1	1
SW	1	0	x	00 (+)	1	0	1	0	0	x
BEQ	1	0	x	01 (-)	0	1	0	0	0	x
J	x	1	x	XX	x	x	0	0	0	x

Observații:

- Semnalele de control sunt generate în etajul ID de unitatea **Main Control**.
- Semnalele de control trebuie să se propage prin etaje deodată cu datele, astfel:
 - până în EX: **RegDST**, **ALUOp**, **ALUSrc**;
 - până în MEM: **Branch**, **MemWrite**, **MemRead**;
 - până în WB: **RegWrite**, **MemtoReg**.



8.2.3 Arhitectura completă – date și control



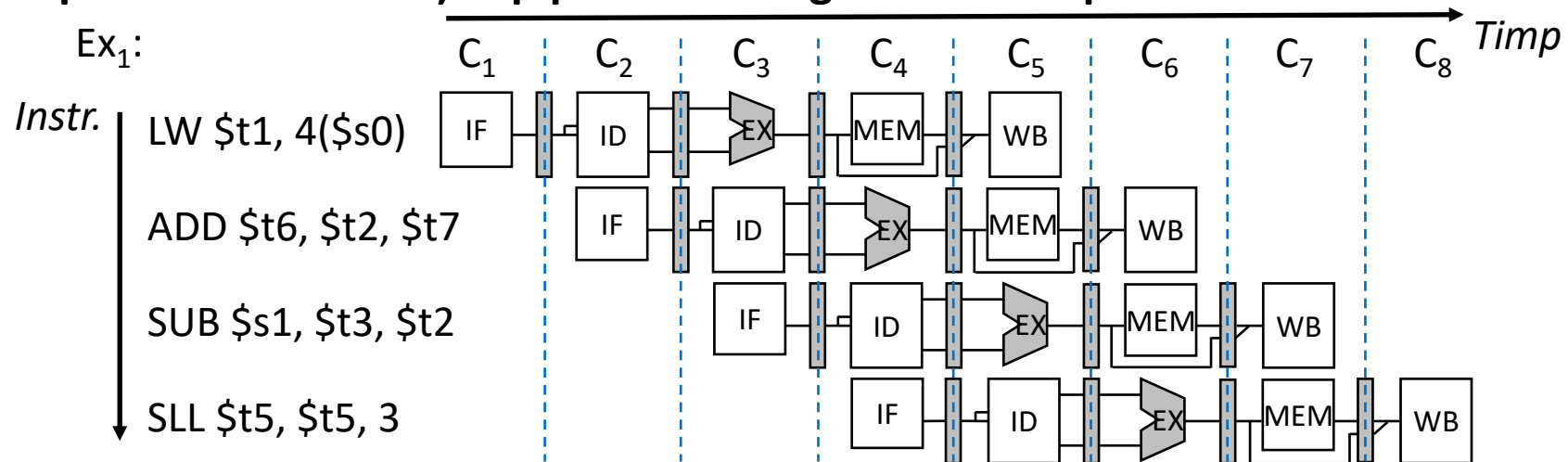
Arhitectura MIPS pipeline completă

8.3 Hazarduri

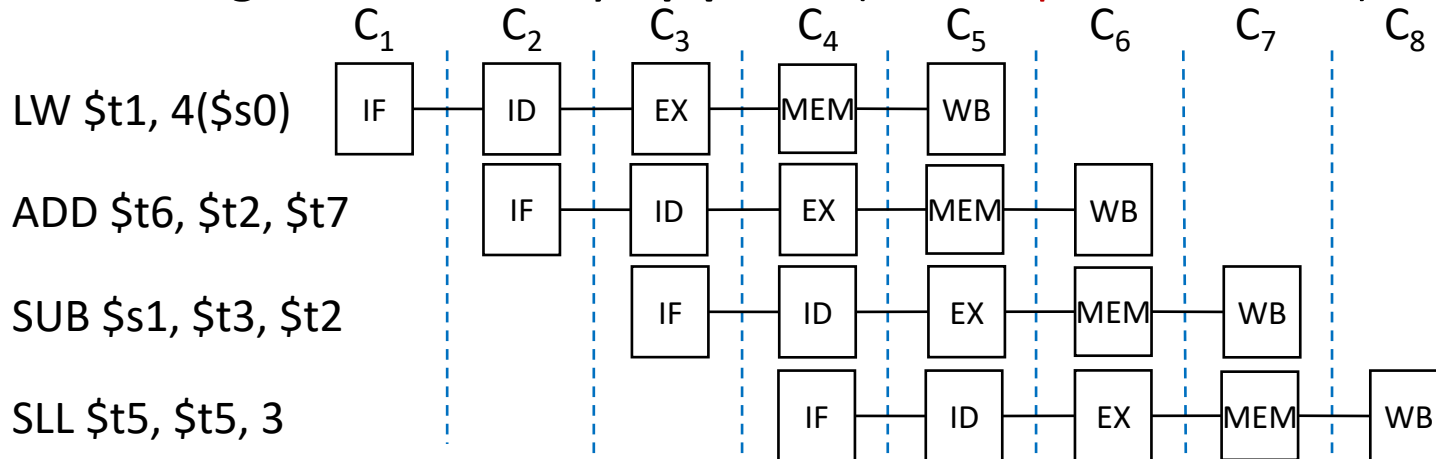
Tipuri de hazard

- structural
- de date
- de control

Reprezentarea execuției pipeline cu diagrame de timp

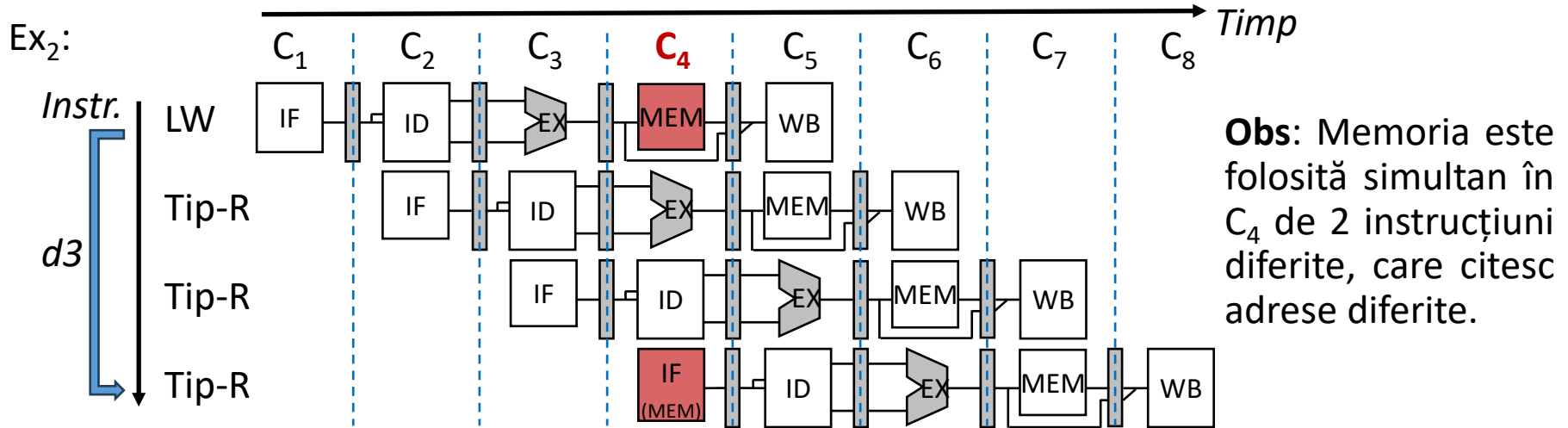


Alternativă a diagramei de execuție pipeline (mai simplu de desenat)

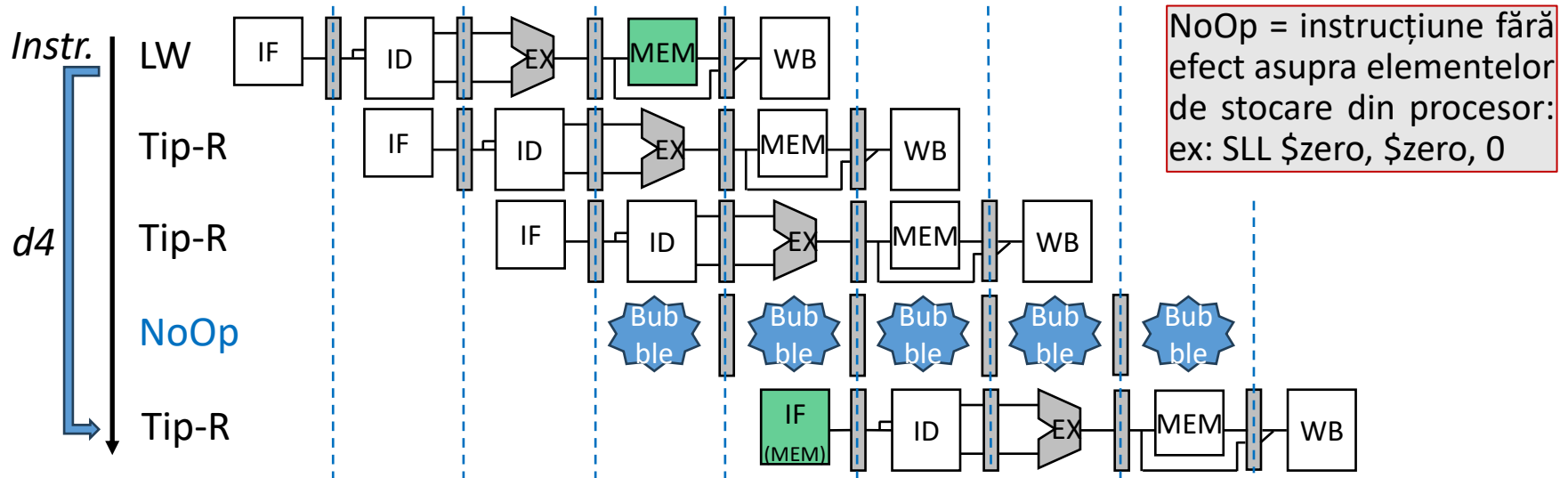


8.3 Hazarduri

8.3.1 Hazard **structural** → Presupunem instr. și datele într-o memorie comună.



Soluție₁: Întârziere (Stall) prin inserare NoOp (de compiler) => pe etaje se propagă un **Bubble**.

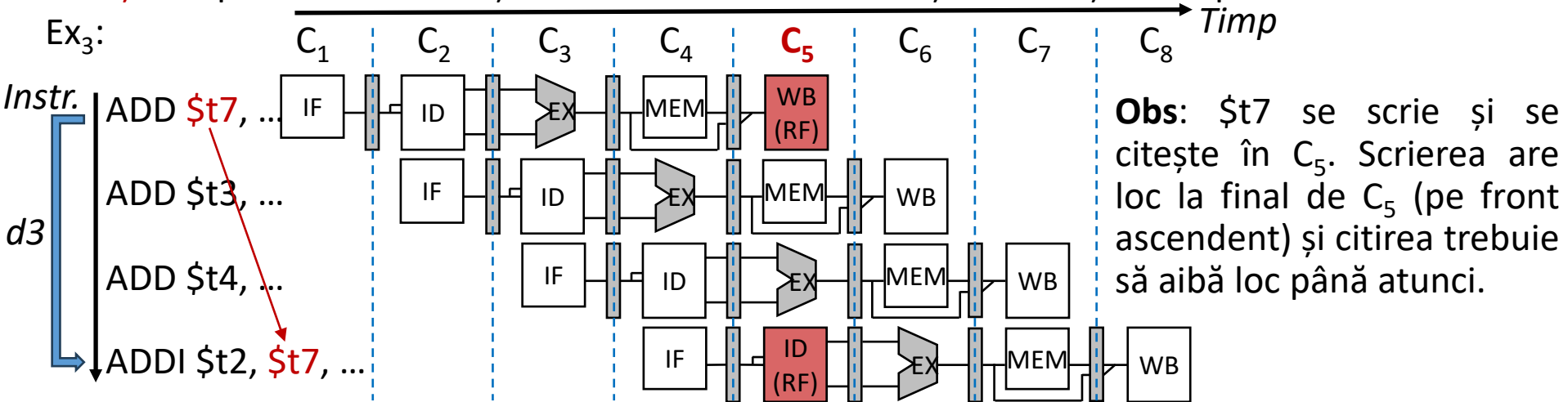


Soluție₂ (fără întârziere): **Memorii separate** => **Memoria de Instrucțiuni** și **Memoria de Date**.

8.3 Hazarduri

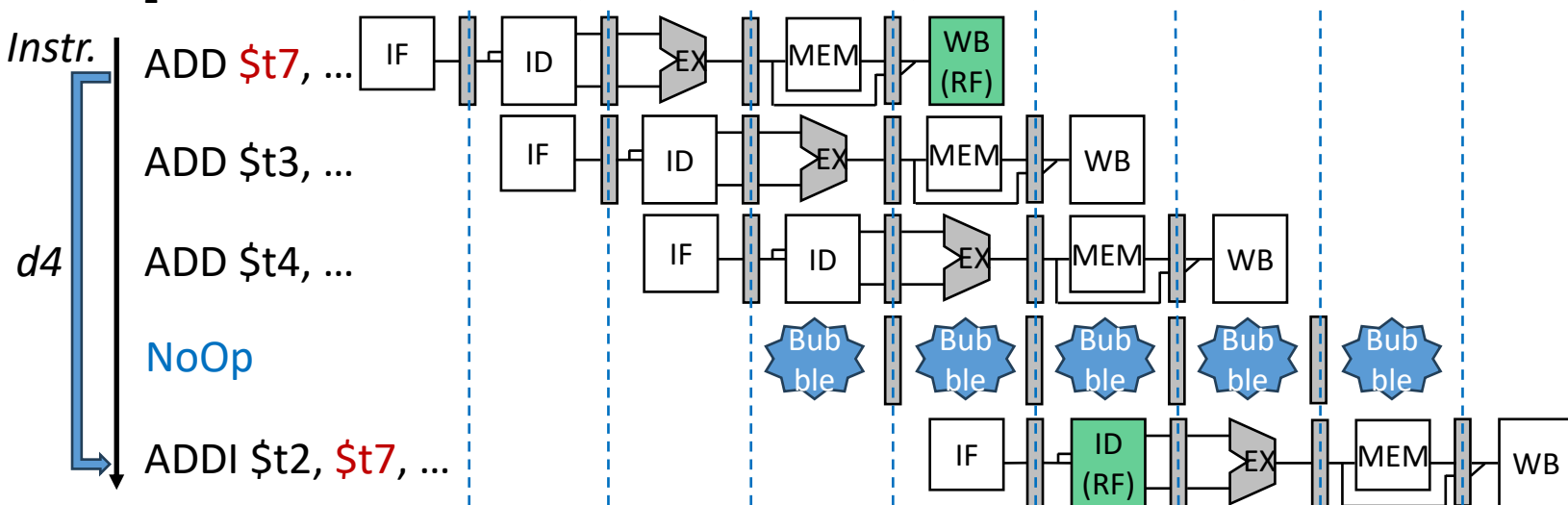
8.3.1 Hazard structural (cont) → Considerăm instr. și datele în memorii separate.

Definiție = apare când instrucțiuni diferite folosesc aceeași resursă și nu se pot executa simultan.



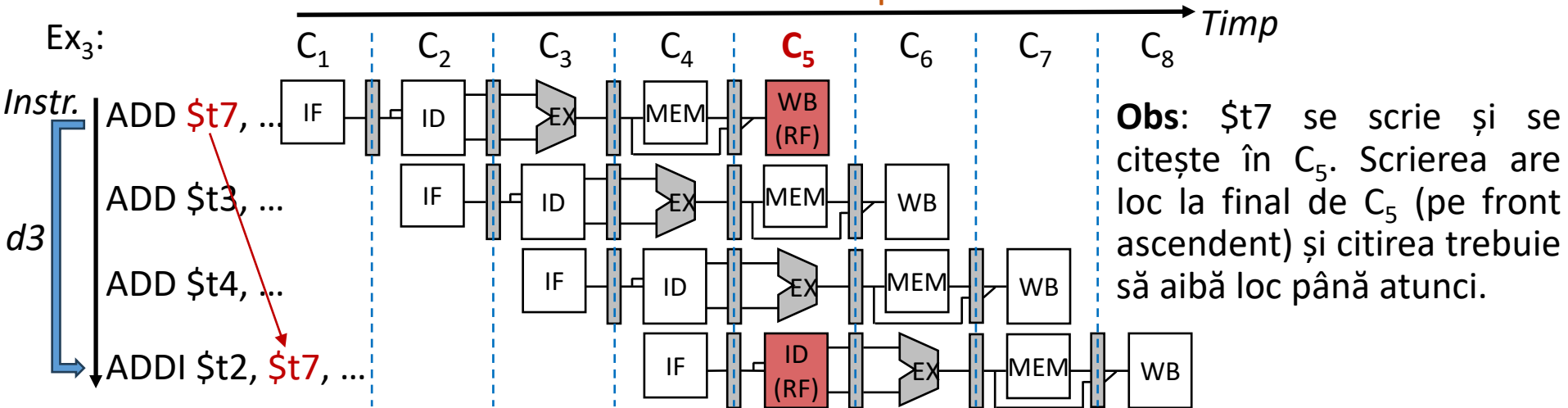
Notă: Deși încadrat ca hazard structural, această situație este de fapt un hazard de date. 🤔

Soluție₁ (*software*): **Întârziere** prin inserare NoOp (la nivel de compilator).



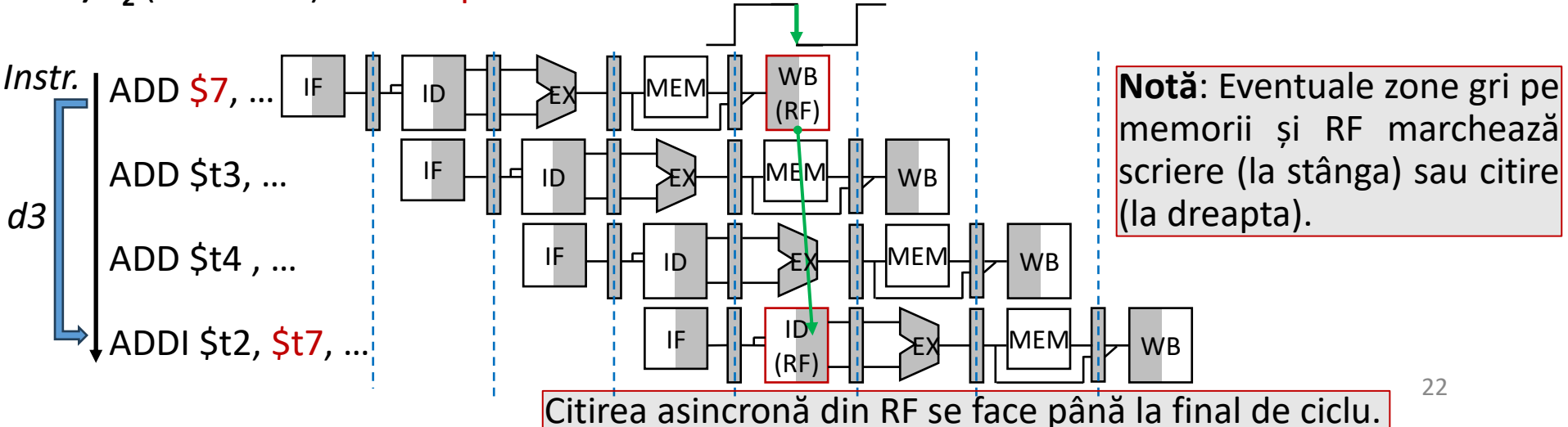
8.3 Hazarduri

8.3.1 Hazard structural (cont) → De acum considerăm instrucțiunile și datele în memorii separate.



Notă: Deși încadrat ca hazard structural, această situație este de fapt un hazard de date. 🤔

Soluție₂ (hardware): Scriere pe frontul descendent în RF => nu necesită întârziere.

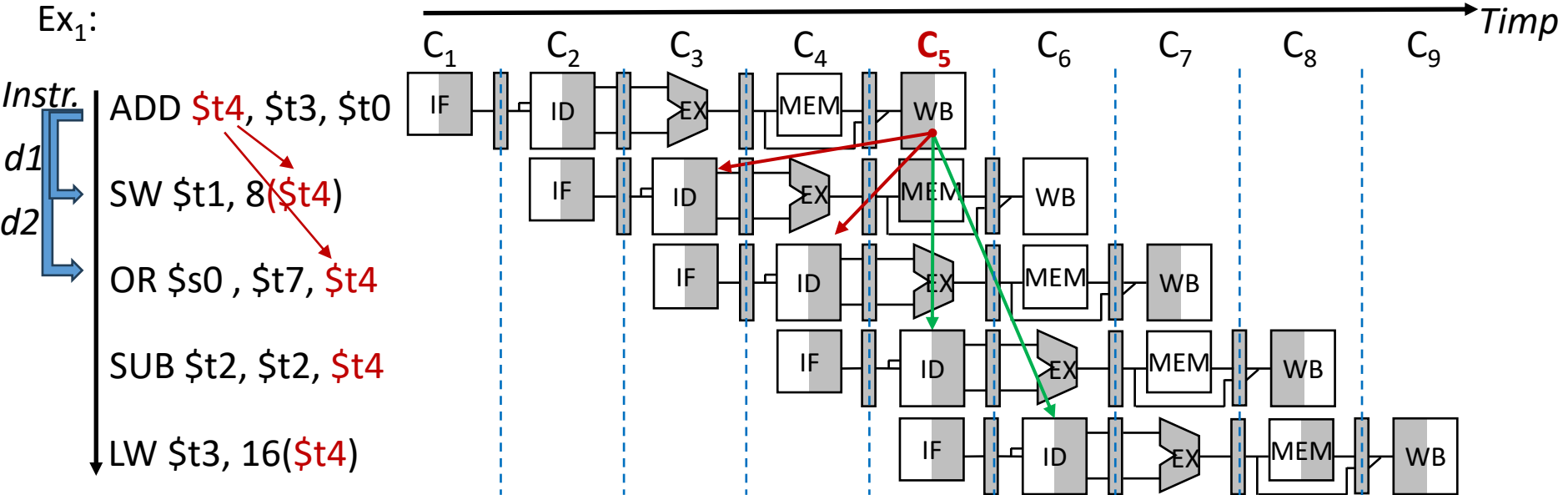


8.3 Hazarduri

8.3.2 Hazard de date

Definiție = apare când sunt folosiți operanzi a căror valoare este în curs de prelucrare în alte etaje pipeline.

Ex₁:



Hazarduri de date = dependențe înapoi în diagramă. Noua valoare a lui $\$t4$ este necesară la SW și OR înainte de ciclul C₅.

Obs:

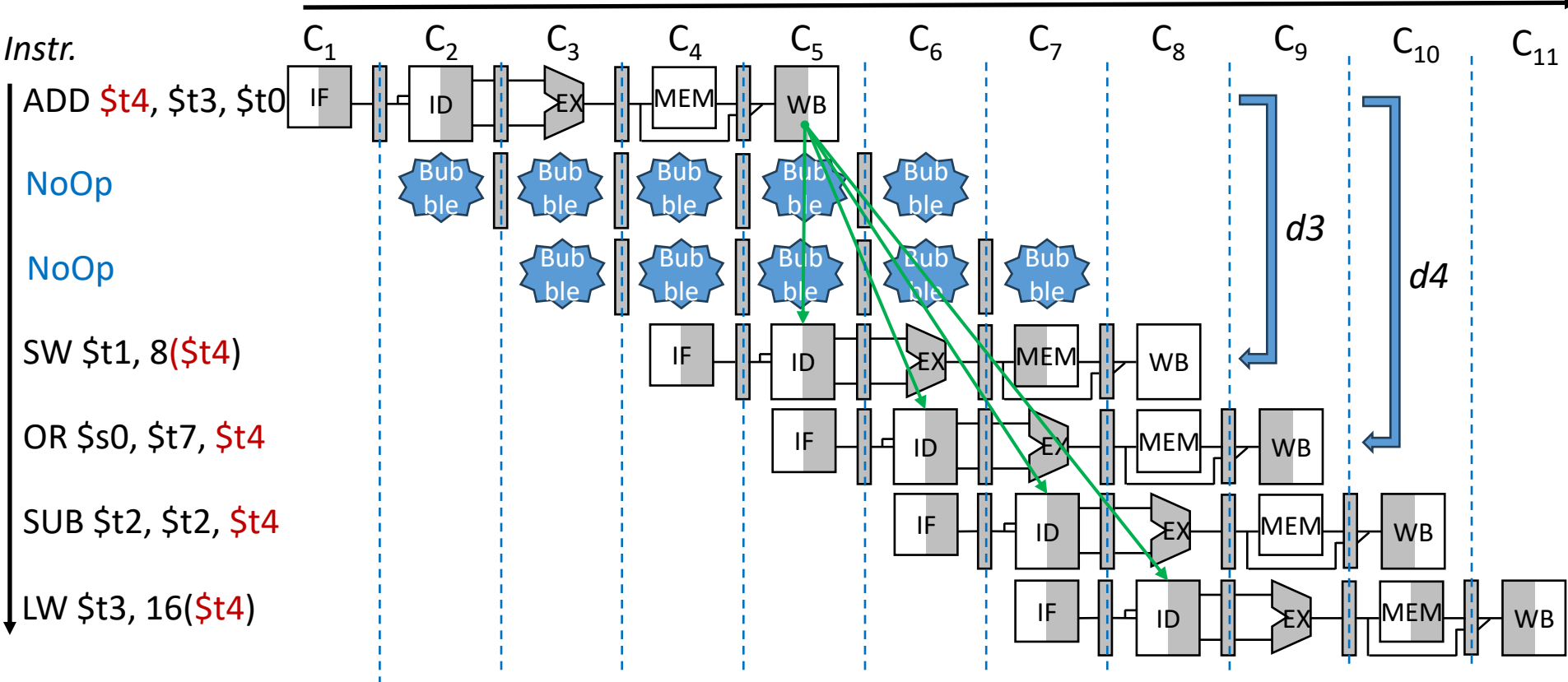
1. Instrucțiunile SW și OR, la distanță de 1 și 2 instrucțiuni de ADD, generează hazard de date pe $\$t4$: citesc registrul în ciclurile C₃ și C₄, înainte să fie scris cu rezultatul sumei (în ciclul C₅).
2. Instrucțiunea SUB, aflată la 3 instrucțiuni de ADD, rezolvă hazardul la registrul $\$t4$ prin scrierea rezultatului în RF, pe frontul descendent, în ciclul C₅.
3. Instrucțiunea LW, aflată la 4 instrucțiuni de ADD, nu generează hazard.

Concluzie: Hazardurile de date apar într-o vecinătate de până la 3 instrucțiuni consecutive.

8.3 Hazarduri

8.3.2 Hazard de date (cont)

Ex₁ – Soluție₁ (*software*): *Întârziere* prin inserare a 2 NoOp între ADD și SW (*de compilator*). *Time*



Obs: Rezolvarea unui hazard de date poate avea ca și consecință rezolvarea simultană a mai multor hazarduri.

Avantaj: Se poate realiza ușor prin software, la nivel de compilator (NoOp = SLL \$zero, \$zero, 0).

Dezavantaj: Poate apărea des și introduce întârziere în execuție => reduce viteza.

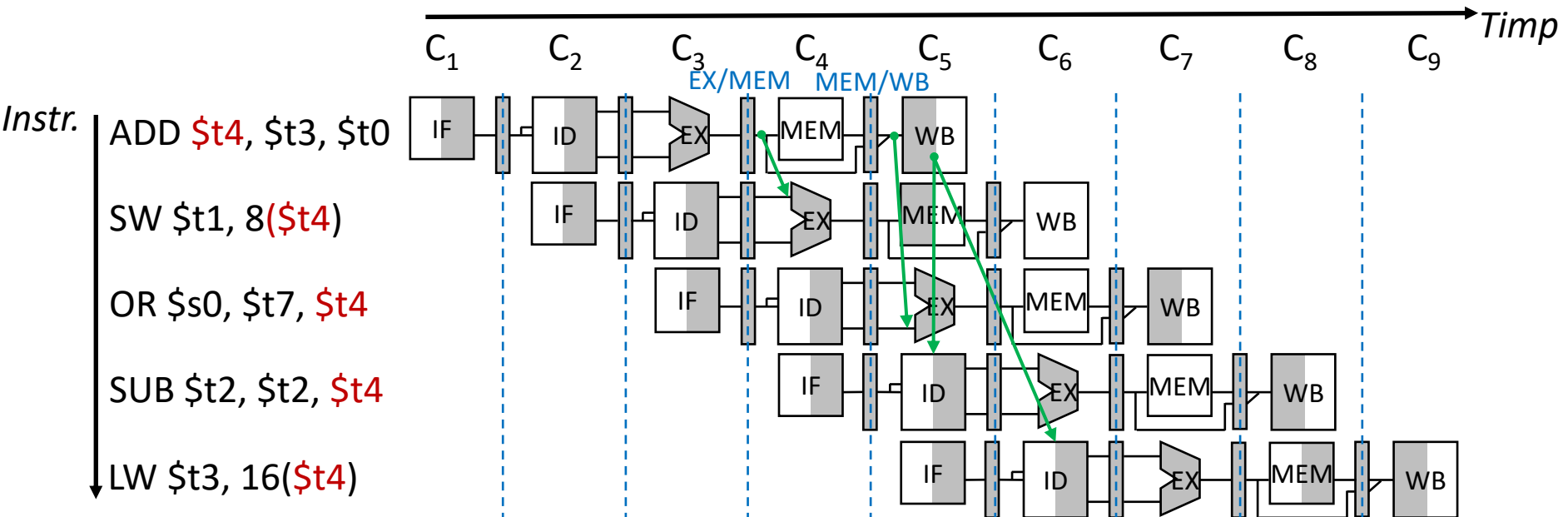
8.3 Hazarduri

8.3.2 Hazard de date (cont)

Ex₁ – Soluție₂ (*hardware*): **Înaintarea datelor** (forwarding) prin tehnici hardware.

Forwarding = folosirea rezultatelor din etajele EX sau MEM înainte de scrierea lor în registre.

Se folosesc registrele pipeline **EX/MEM** și **MEM/WB** la înaintarea valorii calculate pentru **\$t4**.

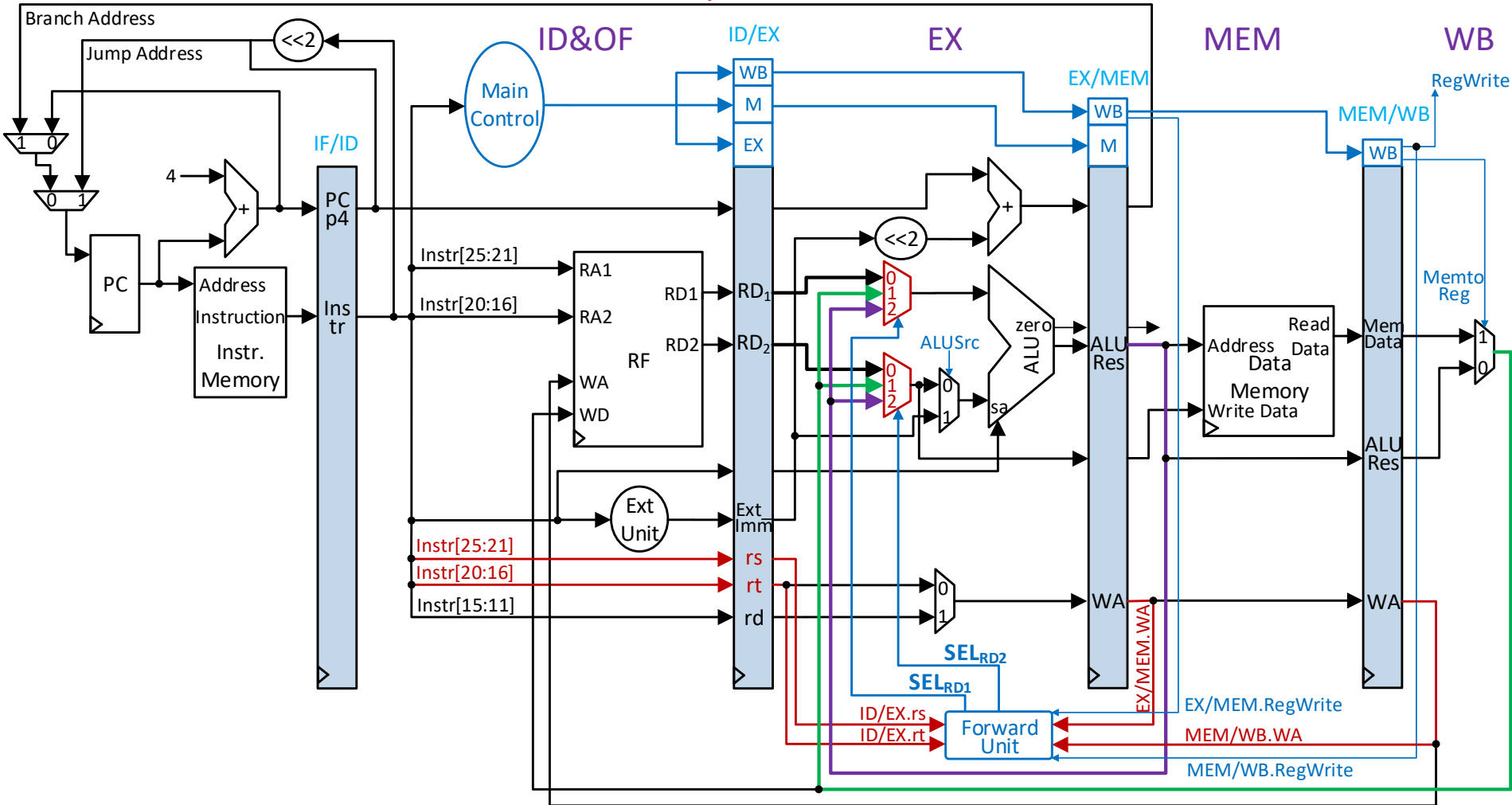


Obs:

- Deși noua valoare pentru **\$t4** se calculează în ciclul C₃ (etajul EX), înaintarea are loc în ciclurile următoare (MEM și WB), când valoarea apare salvată în registrele pipeline.
- Când înaintarea se realizează din etajul MEM spunem că avem **hazard de date MEM**, iar când se realizează din etajul WB spunem că avem **hazard de date WB**.

8.3 Hazarduri

8.3.2 Hazard de date – Hardware pentru înaintare



- Se introduc două MUX 4:1 pentru selectarea operanzilor ALU, cu sursele: 00_2 – intrarea din RF; 01_2 – **date înainte de la etajul WB**; 10_2 – **date înaintate de la MEM**; 11_2 – x.
- **Forward Unit** detectează hazardul pe baza câmpurilor **ID/EX.rs**, **ID/EX.rt**, **EX/MEM.WA**, **MEM/WB.WA** și a semnalelor de control **EX/MEM.RegWrite**, **MEM/WB.RegWrite**.

8.3 Hazarduri

8.3.2 Hazard de date – Logica de control pentru înaintare

Condițiile pentru **hazard de date MEM**:

- Instrucțiunea anterioară din etajul MEM **va scrie rezultatul în RF**, în **alt registru decât \$zero** (nu se face înaintare pentru registrul \$zero).
- Scrierea în RF se va face în unul din operanzii necesari în etajul EX => unul din câmpurile **rs sau rt din etajul EX este identic cu câmpul WA din etajul MEM**.

Selecție pentru înaintare din etajul MEM:

$(\text{EX/MEM.RegWrite} \ \& \ \text{EX/MEM.WA} \neq 0 \ \& \ \text{EX/MEM.WA} = \text{ID/EX.rs}) \rightarrow \text{SEL}_{\text{RD1}} \leftarrow 10 \ (\text{EX/MEM.ALUres})$
 $(\text{EX/MEM.RegWrite} \ \& \ \text{EX/MEM.WA} \neq 0 \ \& \ \text{EX/MEM.WA} = \text{ID/EX.rt}) \rightarrow \text{SEL}_{\text{RD2}} \leftarrow 10 \ (\text{EX/MEM.ALUres})$

Condițiile pentru **hazard de date WB**:

- Instrucțiunea anterioară din etajul WB **va scrie rezultatul în RF**, în **alt registru decât \$zero** (nu se face înaintare pentru registrul \$zero).
- Scrierea în RF se va face în unul din operanzii necesari în etajul EX => unul din câmpurile **rs sau rt din etajul EX este identic cu câmpul WA din etajul WB**.

Selecție pentru înaintare din etajul WB:

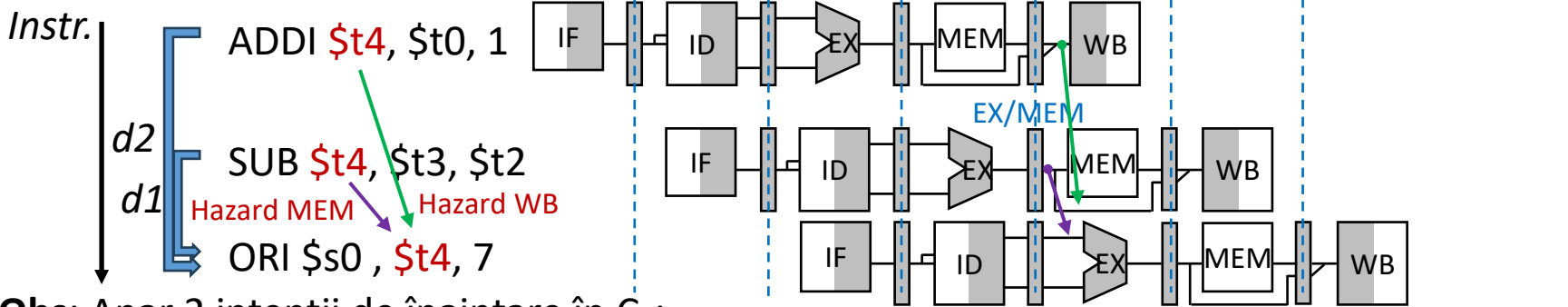
$(\text{MEM/WB.RegWrite} \ \& \ \text{MEM/WB.WA} \neq 0 \ \& \ \text{MEM/WB.WA} = \text{ID/EX.rs}) \rightarrow \text{SEL}_{\text{RD1}} \leftarrow 01 \ (\text{înaintare din WB})$
 $(\text{MEM/WB.RegWrite} \ \& \ \text{MEM/WB.WA} \neq 0 \ \& \ \text{MEM/WB.WA} = \text{ID/EX.rt}) \rightarrow \text{SEL}_{\text{RD2}} \leftarrow 01 \ (\text{înaintare din WB})$

8.3 Hazarduri

8.3.2 Hazard de date – Logica de control pentru înaintare (cont)

Situație conflictuală

Ex₂:



Obs: Apar 2 intenții de înaintare în C₅:

1. pentru **hazardul WB** între ADDI și ORI.
2. pentru **hazardul MEM** între SUB și ORI.

Care e cea corectă?

Răspuns: Doar înaintarea rezultatului de la instrucțiunea cea mai recentă (SUB) este corectă.

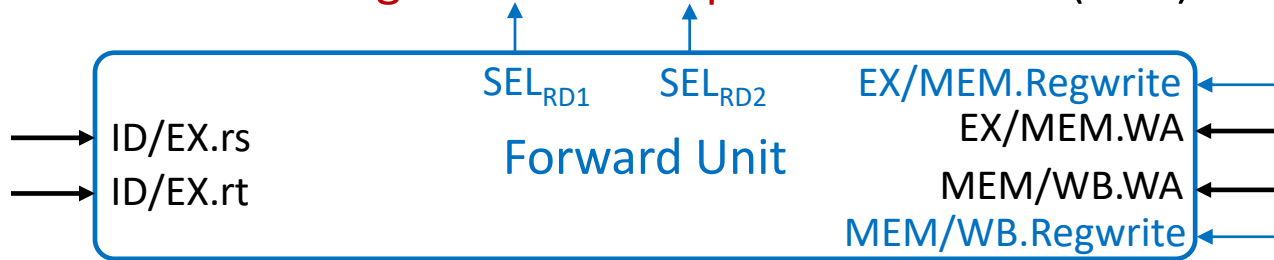
Soluție: Prioritate are înaintarea pentru hazardul MEM => se adaptează condițiile, astfel:

```
if EX/MEM.RegWrite & EX/MEM.WA ≠ 0 & EX/MEM.WA = ID/EX.rs then SELRD1 ← 10 (înaintare din MEM)
else if MEM/WB.RegWrite & MEM/WB.WA ≠ 0 & MEM/WB.WA = ID/EX.rs then SELRD1 ← 01 (înaintare din WB)
else SELRD1 ← 00 (ID/EX.RD1, fără înaintare)
```

```
if EX/MEM.RegWrite & EX/MEM.WA ≠ 0 & EX/MEM.WA = ID/EX.rt then SELRD2 ← 10 (înaintare din MEM)
else if MEM/WB.RegWrite & MEM/WB.WA ≠ 0 & MEM/WB.WA = ID/EX.rt then SELRD2 ← 01 (înaintare din WB)
else SELRD2 ← 00 (ID/EX.RD2, fără înaintare)
```

8.3 Hazarduri

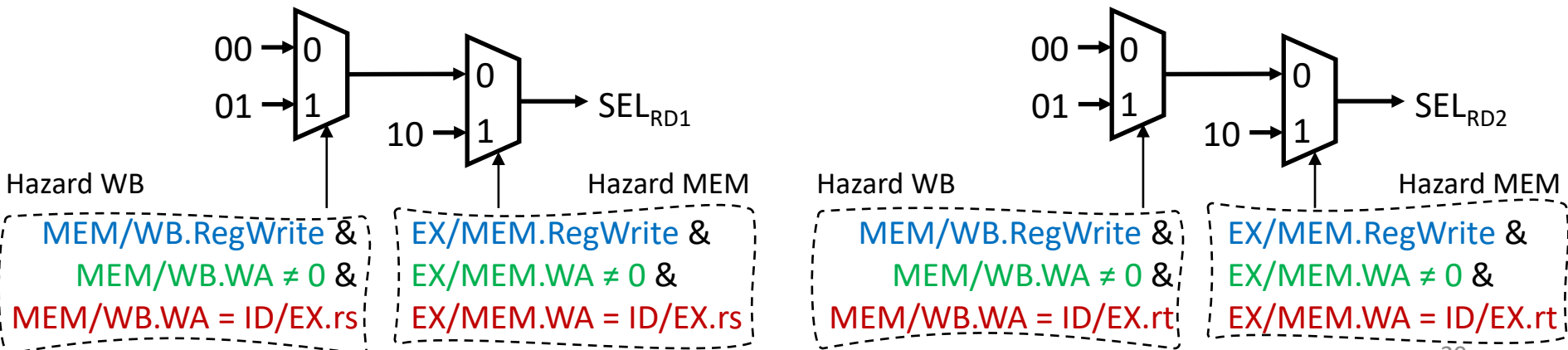
8.3.2 Hazard de date – Logica de control pentru înaintare (cont)



if EX/MEM.RegWrite & EX/MEM.WA $\neq$ 0 & EX/MEM.WA = ID/EX.rs then SEL_{RD1} $\leftarrow$ 10 (înaintare din MEM)
 else if MEM/WB.RegWrite & MEM/WB.WA $\neq$ 0 & MEM/WB.WA = ID/EX.rs then SEL_{RD1} $\leftarrow$ 01 (înaintare din WB)
 else SEL_{RD1} $\leftarrow$ 00 (ID/EX.RD₁)

if EX/MEM.RegWrite & EX/MEM.WA $\neq$ 0 & EX/MEM.WA = ID/EX.rt then SEL_{RD2} $\leftarrow$ 10 (înaintare din MEM)
 else if MEM/WB.RegWrite & MEM/WB.WA $\neq$ 0 & MEM/WB.WA = ID/EX.rt then SEL_{RD2} $\leftarrow$ 01 (înaintare din WB)
 else SEL_{RD2} $\leftarrow$ 00 (ID/EX.RD₂)

Forward Unit – implementare cu MUX

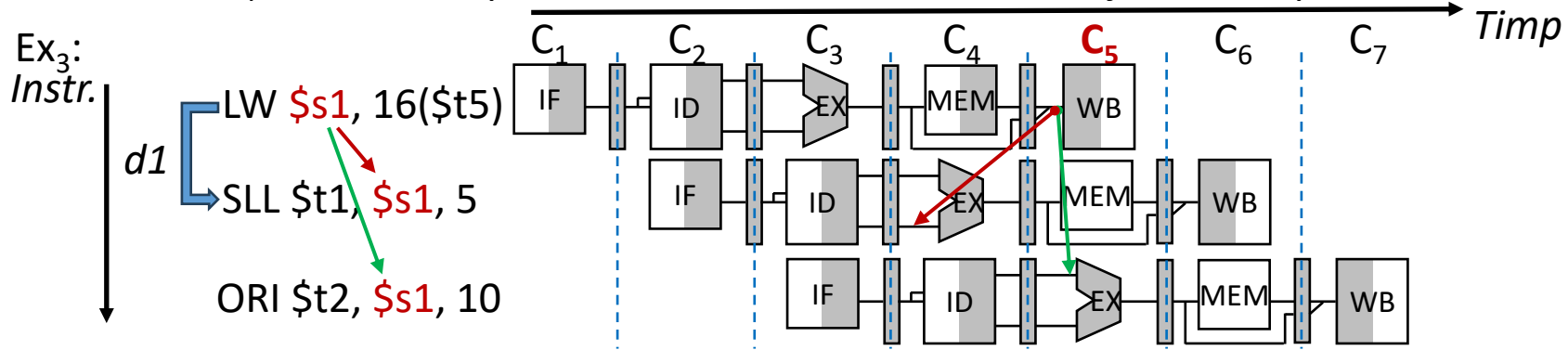


8.3 Hazarduri

8.3.2 Hazard de date – Tehnica de așteptare (stall)

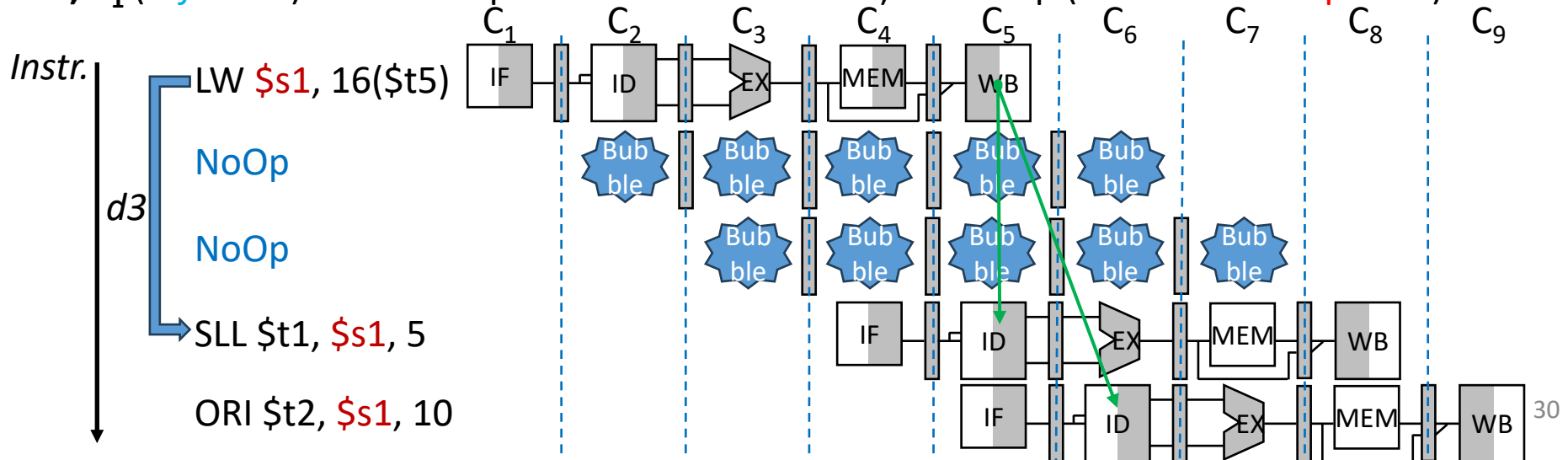
Limitări ale înaintării

Obs: La instrucțiunea LW se poate face înaintare numai din etajul WB, după citirea din memorie.



Problemă: Înaintarea din WB rezolvă hazardul pentru ORI în C₅, dar nu și pentru SLL. Hazardurile provocate de instrucțiuni load se numesc **load data hazard** (sunt tot hazarduri de date).

Soluție₁ (software): Întârziere prin inserare a 2 instrucțiuni NoOp (la nivel de compilator).

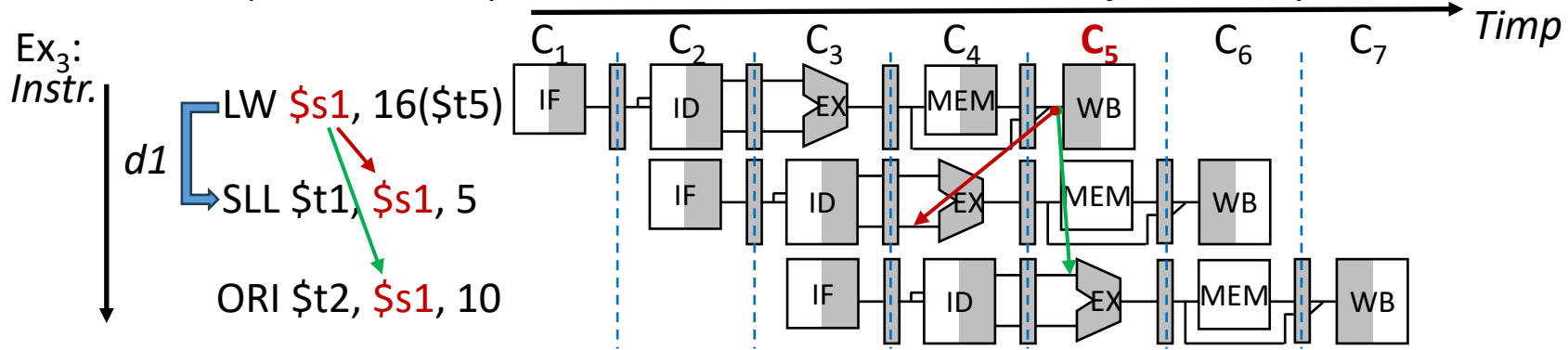


8.3 Hazarduri

8.3.2 Hazard de date – Tehnica de așteptare (stall) (cont)

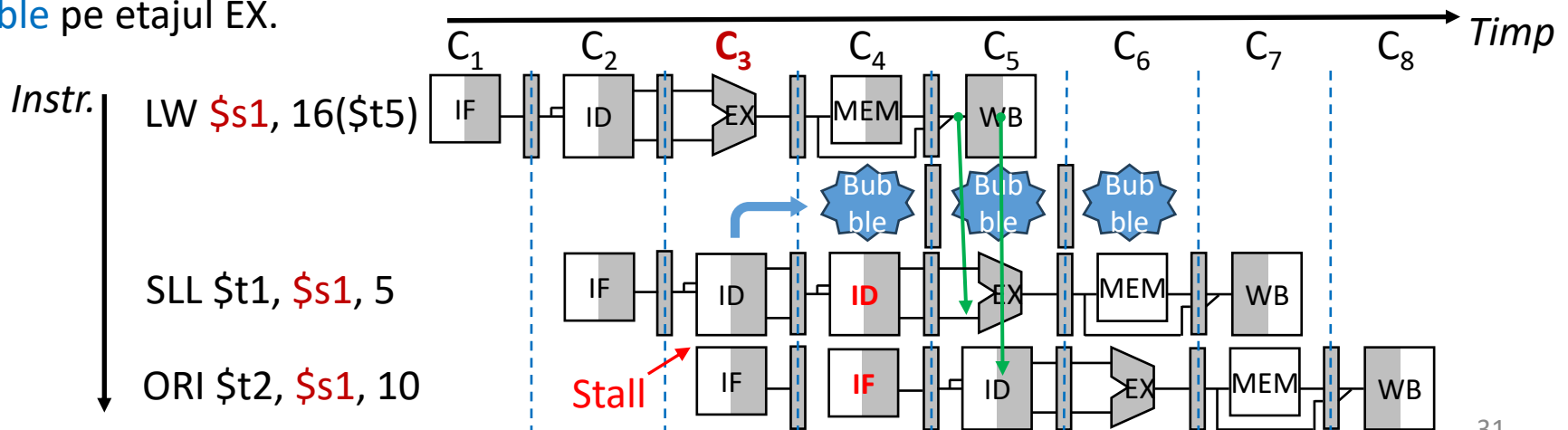
Limitări ale înaintării

Obs: La instrucțiunea LW se poate face înaintare numai din etajul WB, după citirea din memorie.



Problemă: Înaintarea din WB rezolvă hazardul pentru ORI în C₅, dar nu și pentru SLL. Hazardurile provocate de instrucțiuni load se numesc **load data hazard** (sunt tot hazarduri de date).

Soluție₂ (hardware): Blocarea instrucțiunilor SLL și ORI în etajele ID, respectiv IF și inserare **Bubble** pe etajul EX.



8.3 Hazarduri

8.3.2 Hazard de date – Logica de control pentru așteptare (stall)

Se adaugă unitatea **Hazard Detection Unit**: detectează **load data hazard** în ID și inițiază **stall**.

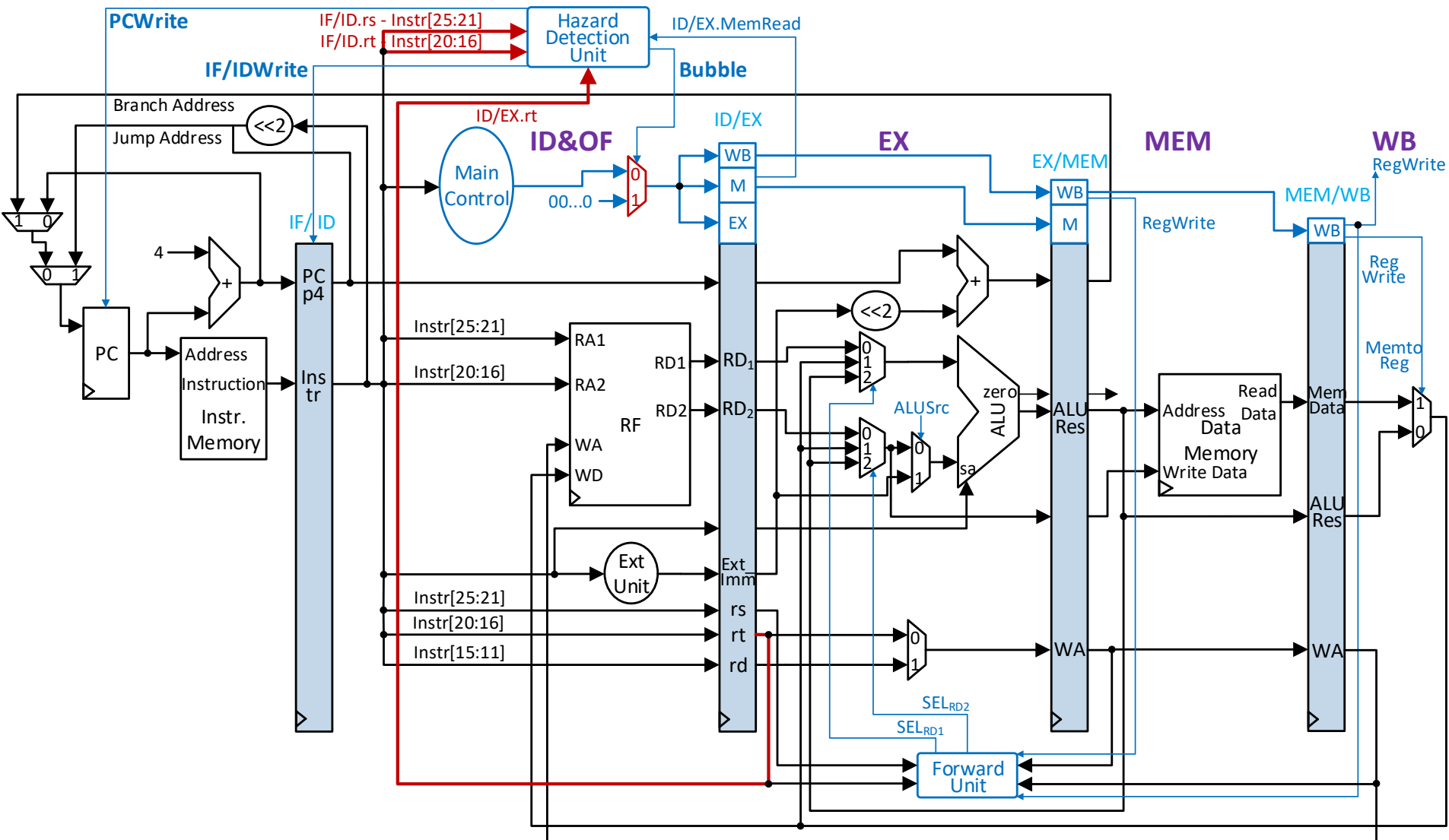
Notă: Dacă hazardul este detectat în etajul ID => instrucțiunea LW este în EX.

Condițiile testate în Hazard Detection Unit:

1. Instrucțiunea LW este în etajul EX. $ID/EX.MemRead = 1$

și 2. LW scrie un registru necesar în ID. $ID/EX.rt = IF/ID.rs \mid ID/EX.rt = IF/ID.rt$
concret $\rightarrow IF/ID.Instr_{25:21} \quad IF/ID.Instr_{20:16}$

8.3.2 Hazard de date – Hardware pentru aşteptare (stall)



Aşteptarea pipeline (stall) este compusă din:

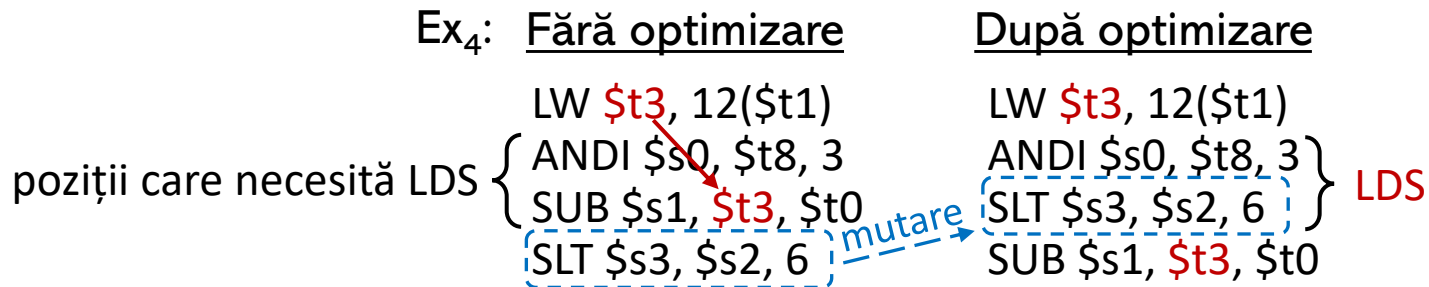
- blocare IF şi ID: $PCWrite \leftarrow 0$, $IF/IDWrite \leftarrow 0$.
- inserare Bubble prin anularea semnalelor de control pentru etajul EX: $Bubble \leftarrow 1$.
 - Notă: Anularea semnalului Jump nu este necesară fiindcă scrierea în PC este oricum blocată de la PCWrite.

8.3 Hazarduri

8.3.2 Hazard de date – Soluții pentru **load data hazard** (abordări uzuale)

➤ **Software** (ex: de către compilator, programator, alte tool-uri de analiză):

1. Inserare 1 sau 2 **instrucțiuni NoOp**, în funcție de situație;
2. Căutare **instrucțiuni independente de LW**, care să ocupe cele 2 poziții aflate imediat după LW, fără a afecta corectitudinea programului. Instrucțiunile independente de LW, aflate pe cele 2 poziții, poartă denumirea de **Load-Delay Slots (LDS)**. Dacă nu se găsesc suficiente, se completează cu NoOp.



➤ **Hardware**:

- Combinație între **Așteptare** (Stall) și **Înaintare** (Forwarding) => reduce numărul de întârzieri cu 1 față de varianta software (1.) cu inserare instrucțiuni NoOp.

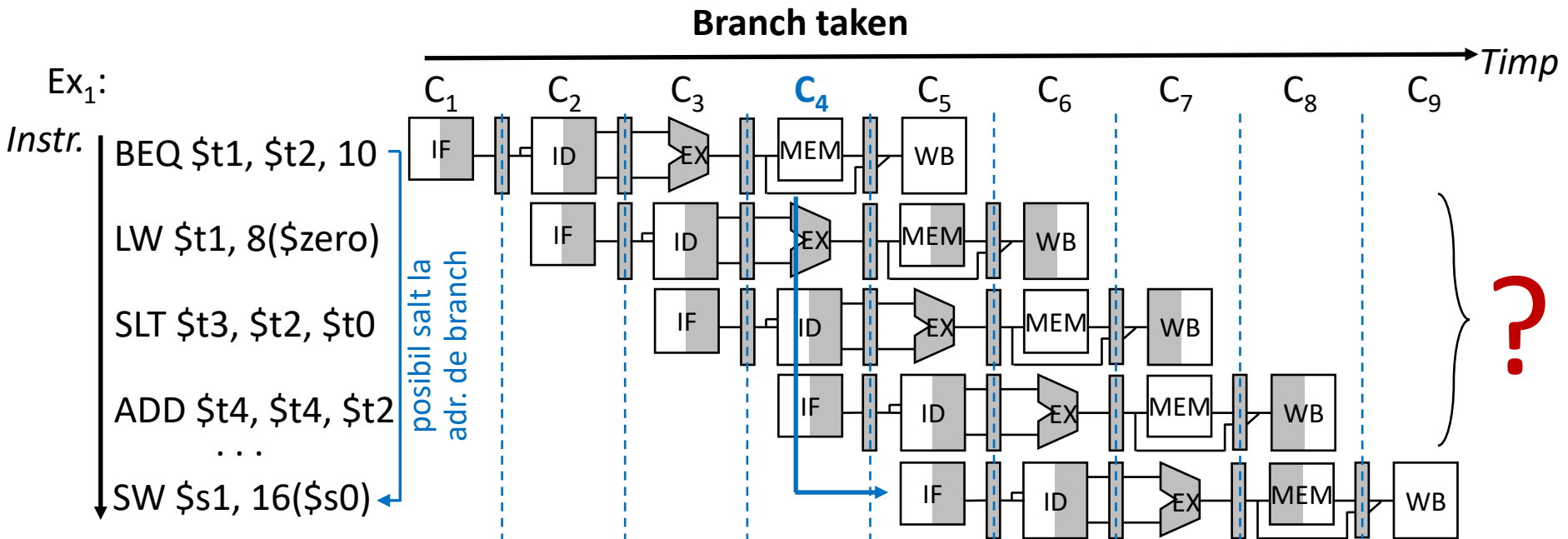
8.3 Hazarduri

8.3.3 Hazard de control

- Apare la instrucțiuni de salt condiționat (branch).
- Se mai numește și **branch hazard**.

Obs: Condiția de salt evaluată în EX poate avea efect abia în etajul MEM. Apar 2 situații posibile:

1. Nu se face salt (**branch not taken**) și execuția continuă nealterată (**corect** ✓);
2. Se face salt (**branch taken**), dar se execută și instrucțiunile din etajele anterioare (**incorect** ✗).



Definiție = la branch, hazardul de control apare deoarece nu se cunoaște cu exactitate următoarea instrucțiune ce trebuie încărcată în etajul IF.

8.3.3 Hazard de control (cont)

Diagram illustrating a branch misprediction scenario in a 5-stage processor (IF, ID, EX, MEM, WB) over 11 clock cycles (C₁ to C₁₁). The instruction sequence is:

- BEQ \$t1, \$t2, ? (Branch instruction, mispredicted)
- NoOp
- NoOp
- NoOp
- LW \$t1, 8(\$zero)
- SLT \$t3, \$t2, \$t0
- ADD \$t4, \$t4, \$t2

The diagram shows the execution of these instructions across the stages. The BEQ instruction is mispredicted, leading to a branch misprediction penalty (Bub ble) in the subsequent cycles. The processor continues to execute the instructions in the pipeline, with the branch misprediction penalty (Bub ble) occurring in the subsequent cycles. The diagram also shows the execution of the subsequent instructions (LW, SLT, ADD) and the branch misprediction penalty (Bub ble) in the subsequent cycles.

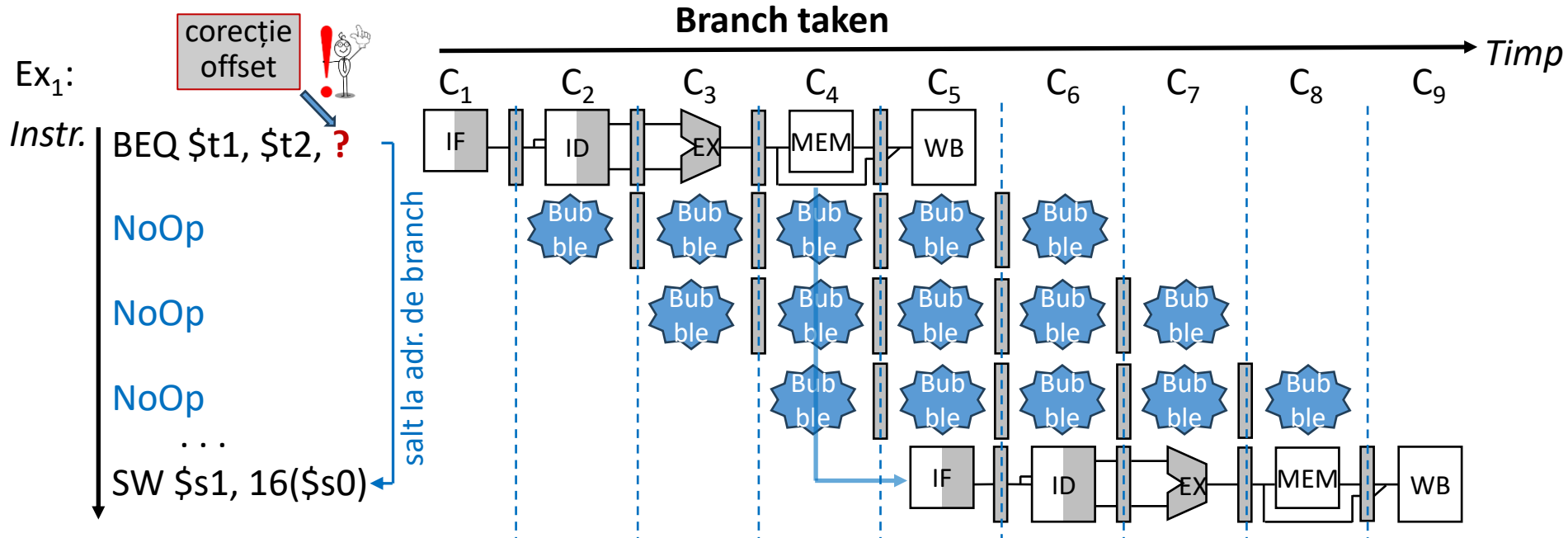
Annotations:

- corecție offset (Correction offset)
- Branch not taken
- Time

8.3 Hazarduri

8.3.3 Hazard de control (cont)

Soluție₁ (*software*): **Întârziere** prin inserare a 3 NoOp (**de către compilator**) după BEQ.



Analiză: SPECint Benchmark execută branch-uri în proporție de 17% din totalul de cicluri. Media CPI (cycles per instruction) la alte instrucțiuni este 1.

Considerând o întârziere de 3 cicluri la 17% din cicluri => $CPI_{\text{pipeline}} = 1 + 3 \times 0.17 = 1.51$ ⇔ deteriorare a productivității la $1/1.51 = 66\%$ => **Dezavantaj**.

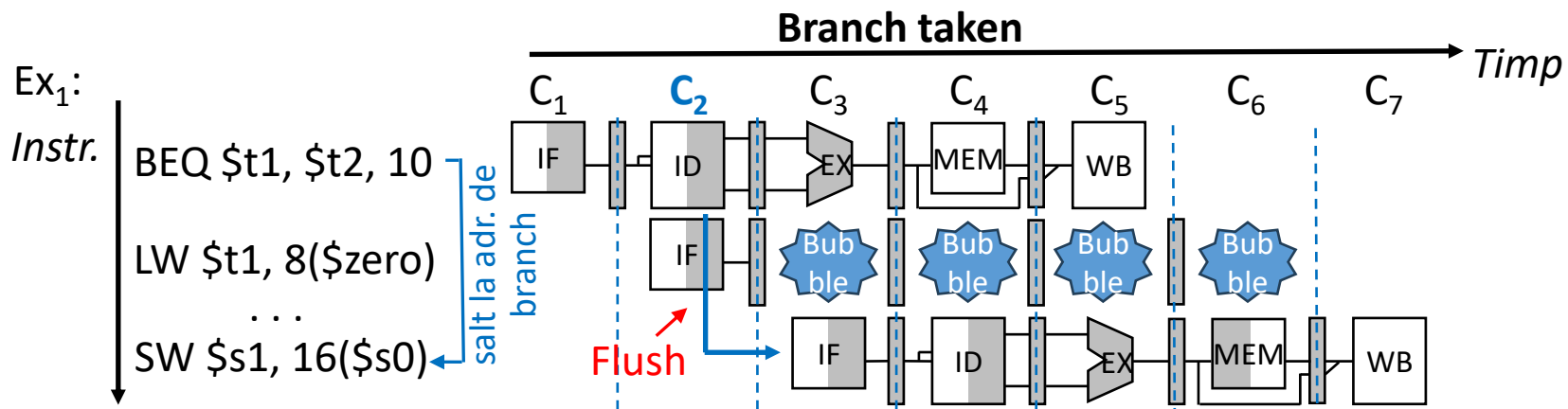
8.3 Hazarduri

8.3.3 Hazard de control (cont)

Soluție₂ (*hardware*): **Predicție** a saltului cu execuție speculativă pentru a reduce frecvența anulării instrucțiunilor. Există mai multe scheme de predicție. Cea mai simplă schemă este **predict not taken**: se presupune că nu se face salt, așadar se încarcă natural în pipeline cele 3 instrucțiuni de după branch. Apar 2 situații în etajul MEM (ciclul C_4) :

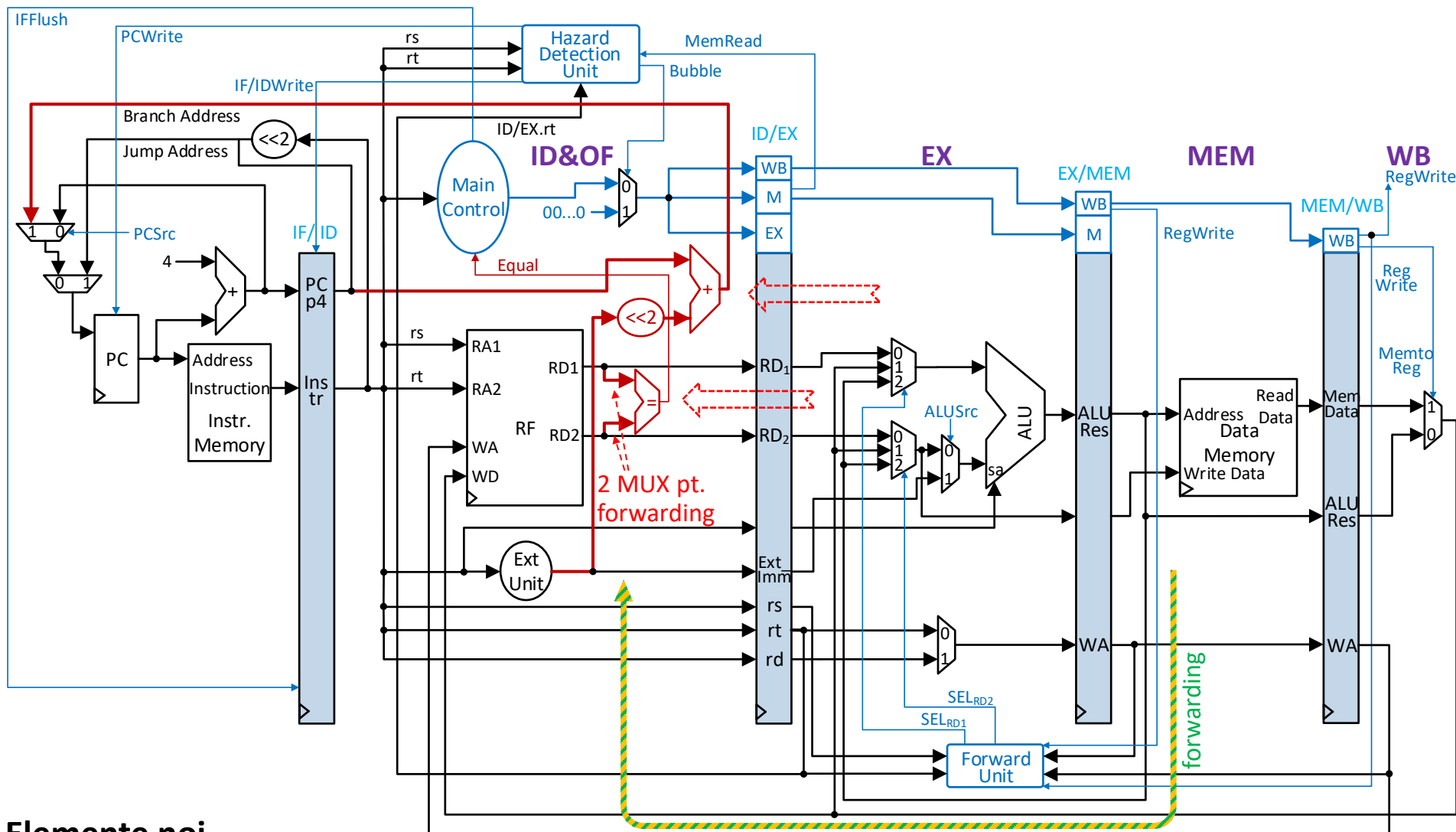
1. Predicția a fost corectă => continuă execuția secvențială, nu apar întârzieri;
2. Predicția a eșuat => se face salt și se anulează (**Flush**) cele 3 instrucțiuni încărcate în pipeline
⇔ 3 cicluri pierdute = **penalitate mare**.

Îmbunătățire: Mutarea deciziei de salt și a calculului adresei de branch în etajul ID => se va anula (**Flush**) doar instrucțiunea din etajul IF ⇔ 1 ciclu pierdut = **penalitate mai mică**.



Obs: Anularea instrucțiunii din IF se face prin resetarea registrului IF/ID => se va genera un **Bubble** (NoOp) în etajul ID, la ciclul următor (C_3).

8.3.3 Hazard de control – Hardware pentru branch în ID și pentru Flush



Elemente noi

- Calculul adresei și testul de egalitate sunt mutate în etajul ID.
- Semnalul **IFFlush** resetează registrul IF/ID: $IFFlush = PCSrc \leftarrow (\text{Branch} \ \& \ \text{Equal})$.

Important: Poate apărea hazard de date pe \$rs și \$rt => se face înaintare din etajul MEM.

Obs: Lipsesc din schemă **2 unități MUX** pentru **forwarding** din etajul MEM.

8.3 Hazarduri

8.3.3 Hazard de control – Ramificare întârziată (delayed branch)

Întârzierea unui branch n = nr. de cicluri (etaje) după IF până se realizează decizia și saltul =>

- la branch în ID avem $n=1$;
- la branch în MEM avem $n=3$;
- există arhitecturi cu $n>3$ => penalitatea **Flush** crește. **Soluție**: alternative software pentru eliminarea evenimentelor de **Flush**.

Branch Delay Slot (BDS) = instrucțiune aflată imediat după branch, a cărei execuție nu e afectată de decizia branch. Întârzierea n indică numărul necesar de BDS pentru a evita **Flush**.

Soluție₃ (*software*): **Căutare** instrucțiuni BDS (**de către compilator**) (similar cu Load Delay Slots la **load data hazard**). Dacă nu se găsesc suficiente, se completează cu NoOp.

Sursa instrucțiunilor alese ca BDS:

- Dinainte de instrucțiunea branch – from **before** (instrucțiuni preferate deoarece nu depind de decizia branch; dacă nu se găsesc suficiente instrucțiuni, se caută cu variantele b și c);
- Imediat după instrucțiunea branch – from **fall-through** (necesită o analiză complexă);
- Începând cu adresa la care se face saltul – from **target** (necesită o analiză complexă).

Ex₂ (cu branch în ID): Fără optimizare

poziție care necesită BDS {
ORI \$t2, \$zero, 4
BEQ \$t5, \$s1, 6
SUB \$t3, \$zero, \$t3

După optimizare

BEQ \$t5, \$s1, ?
ORI \$t2, \$zero, 4
SUB \$t3, \$zero, \$t3

corecție offset

BDS

8.3 Hazarduri

8.3.3 Hazard de control – salturi necondiționate (jump)

Obs: Jump se realizează în etajul ID $\Leftrightarrow$ are întârzierea $n=1$.

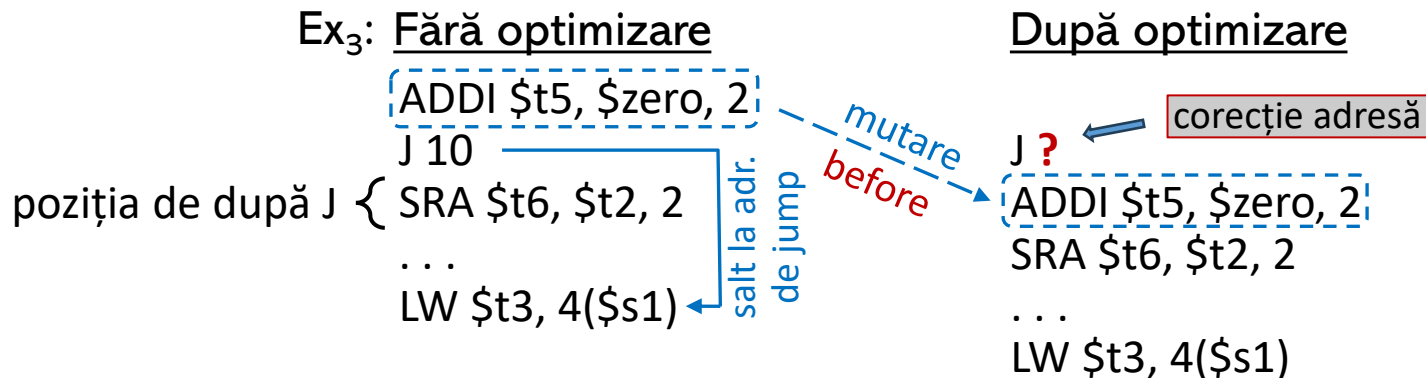
Soluții:

➤ **Software** (ex: de către compilator, programator, alte tool-uri de analiză):

1. Căutare **instrucțiune dinainte de Jump**, care să ocupe poziția de după Jump.

Condiții:

- a) Să nu fie instrucțiuni de salt (alt branch sau jump);
- b) Să nu se facă salt la ea;
- c) Să nu introducă hazarduri cu instrucțiunile de la adresa de salt.



2. Inserare **1 instrucțiune NoOp** (*necesită corecția adresei de salt, dacă e cazul*).

➤ **Hardware:**

1. **Flush**; $\text{IFFlush} \leftarrow \text{Jump} \mid (\text{Branch \& Equal})$

sau 2. Realizare **salt în etajul IF** – necesită hardware dedicat, dar evită **Flush**.

8.4 Hazarduri – Sumar pentru laborator

Soluții pentru laborator: Inserare de instrucțiuni NoOp.

➤ Hazard structural



Notă: După inserare, poate fi necesară actualizarea adreselor de jump și a offset-urilor de branch conform cu noile locații.

- Apare la distanță de 3 instrucțiuni (o instrucțiune scrie și a 3-a după ea citește același registru);
- **Soluție:** Scriere pe frontul descendent (rezolvare implicită).

➤ Hazard de date

- Apare la distanță de 1 sau 2 instrucțiuni (o instrucțiune scrie și prima sau a 2-a după ea citește același registru);
- **Soluție:** Se inserează 2, respectiv 1 NoOp între instrucțiuni, pentru a crește distanța la 3.

➤ Hazard de control

- Hazard introdus de branch:
 - **Soluție:** Se inserează 3 instrucțiuni NoOp după instrucțiunea de branch.
- Hazard introdus de jump:
 - **Soluție:**
 1. Schimb între jump și instrucțiunea anterioară, dacă îndeplinește cele 3 condiții:
 - a) Nu este instrucțiune de salt (alt branch sau jump);
 - b) Nu se face salt la ea;
 - c) După schimbare, nu introduce hazarduri cu instrucțiunile de la adresa de salt.
 2. Dacă nu funcționează soluția 1, atunci se inserează un NoOp după jump.

Probleme propuse

1. Implementarea altor instrucțiuni în pipeline (vezi Curs 4).

2. Se cere diagrama de execuție pipeline, calcul CPI/iterație și numărul de cicluri pe primele 2 iterații, la secvențele alăturate, în diverse condiții:

- branch în etajul ID sau MEM;
- cu și fără Forwarding și Stall.

loop: LW \$t6, 4000(\$t7)
 ADD \$t9, \$t6, \$t3
 OR \$t5, \$t9, \$t6
 LW \$t2, 2000(\$t5)
 ADD \$t3, \$t9, \$t2
 SUBI \$t5, \$t5, 12
 SW \$t9, 2000(\$t3)
 BNE \$t9, \$zero, loop

loop: LW \$t0, A(\$t2)
 LW \$t1, B(\$t2)
 SUBU \$t0, \$t0, \$t1
 SW \$t0, C(\$t2)
 ADDIU \$t2, \$t2, 4
 SUBU \$t5, \$t3, \$t2
 BNEZ \$t5, loop

3. Se cere diagrama de execuție pipeline cu Stall, fără și cu Forwarding, la secvența de mai jos:

ADD \$t1, \$t5, \$t3
 LW \$t2, 0(\$t1)
 ADD \$t1, \$t2, \$t3
 SW \$t1, 0(\$t5)

Fără forwarding													
C ₁	C ₂	C ₃	C ₄	C ₅	C ₆	C ₇	C ₈	C ₉	C ₁₀	C ₁₁	C ₁₂	C ₁₃	C ₁₄
IF	ID	EX	MEM	WB									
	IF	ID	ID _{Stall}	ID _{Stall}	EX	MEM	WB						
		IF	IF _{Stall}	IF _{Stall}	ID	ID _{Stall}	ID _{Stall}	EX	MEM	WB			
					IF	IF _{Stall}	IF _{Stall}	ID	ID _{Stall}	ID _{Stall}	EX	MEM	WB

Cu forwarding									
	C ₁	C ₂	C ₃	C ₄	C ₅	C ₆	C ₇	C ₈	C ₉
ADD	IF	ID	EX	MEM(Fw ₁)	WB				
LW		IF	ID	EX(Fw ₁)	MEM	WB(Fw ₂)			
ADD			IF	ID	ID _{Stall}	EX(Fw ₂)	MEM(Fw ₃)	WB	
SW				IF	IF _{Stall}	ID	EX(Fw ₃)	MEM	WB

Referințe

- [1] D. A. Patterson, J. L. Hennessy, “Computer Organization and Design: The Hardware/Software Interface”, 5th edition, Cap. 4.5-4.8, Ed. Morgan–Kaufmann, 2013.
- [2] MIPS Technologies, Inc., “MIPS32™ Architecture for Programmers, Volume I: Introduction to the MIPS32™ Architecture”.
- [3] MIPS Technologies, Inc., “MIPS32™ Architecture for Programmers Volume II: The MIPS32™ Instruction Set”.
- [4] S. Jayaraman, “Lecture 8: Pipelining: Datapath and Control”, **pg. 42-51, pg. 67-76**, CDA3101: Computer Organization II. Online (material suport cu exemple de execuție în pipeline):
<https://www.cs.fsu.edu/~jayarama/Org2Su18/Slides/PipelinedDatapath.pdf>